

## 4

## Relational Database Design

**Syllabus**

**Relational Model** : Basic concepts, Attributes and Domains, CODD's Rules. **Relational Integrity** : Domain, Referential Integrities, Enterprise Constraints. **Database Design** : Features of Good Relational Designs, Normalization, Atomic Domains and First Normal Form, Decomposition using Functional Dependencies, Algorithms for Decomposition, 2NF, 3NF, BCNF.

**Contents****Part I : Relational Model**

4.1 Basic Concepts

4.2 Attributes and Domains

4.3 CODD's Rules .....Aug.-17

.....Oct.-19 ..... Marks 5

**Part II : Relational Integrity**

4.4 Keys

4.5 Constraints .....May-18, Dec.-19 ..... Marks 5

4.6 Enterprise Constraints

**Part III : Database Design**

4.7 Features of Good Relational Designs .....Dec.-18,

.....Aug.-17 ..... Marks 5

4.8 Data Redundancy and Update Anomalies

4.9 Normalization .....Dec.-1 ..... Marks 5

4.10 Atomic Domains and First Normal Form

4.11 Decomposition using Functional Dependencies .....Oct.-19

.....May-18 ..... Marks 5

4.12 Equivalence and Minimal Cover

4.13 Algorithms for Decomposition

4.14 Lossless Join .....May-18 ..... Marks 6

**4.15 Dependency Preservation****4.16 Second Normal Form (2NF)..... Oct.-19, ..... Marks 7****4.17 Third Normal Form (3NF)..... Oct.-18, 19,****..... Aug.-17,****..... Dec.-18, ..... Marks 7****4.18 BCNF ..... May-18,****..... Dec.-18,19,..... Marks 5****Multiple Choice Questions**

**Part I : Relational Model****4.1 Basic Concepts**

- Relation database is a **collection of tables** having unique names.
- For example - Consider the example of **Student table** in which the information about the student is stored.

| RollNo | Name | Phone      |
|--------|------|------------|
| 001    | AAA  | 1111111111 |
| 002    | BBB  | 2222222222 |
| 003    | CCC  | 3333333333 |

**Fig. 4.1.1 Student table**

The above table consists of three column headers **RollNo**, **Name** and **Phone**. Each row of the table indicates the information of each student by means of his Roll Number, Name and Phone number.

Similarly consider another table named **Course** as follows –

| CourseID | CourseName       | Credits |
|----------|------------------|---------|
| 101      | Mechanical       | 4       |
| 102      | Computer Science | 6       |
| 103      | Electrical       | 5       |
| 104      | Civil            | 3       |

**Fig. 4.1.2 Course table**

Clearly, in above table the columns are **CourseID**, **CourseName** and **Credits**. The **CourseID 101** is associated with the course named **Mechanical** and associated with the course of mechanical there are **4 credit points**. Thus the relation is represented by the table in the relation model. Similarly we can establish the relationship among the two tables by defining the third table. For example – Consider the table **Admission** as

| RollNo | CourseID |
|--------|----------|
| 001    | 102      |
| 002    | 104      |
| 003    | 101      |

**Fig. 4.1.3 Admission table**

From this third table we can easily find out that the course to which the **RollNo 001** is admitted is **computer Science**.

## 4.2 Attributes and Domains

There are some commonly used terms in Relational Model and those are -

**Table or relation** : In relational model, table is a collection of data items arranged in rows and columns. The table cannot have duplicate data or rows. Below is an example of student table

| Roll No | Name | Marks | Phone      |
|---------|------|-------|------------|
| 001     | AAA  | 88    | 1111111111 |
| 002     | BBB  | 83    | 2222222222 |
| 003     | CCC  | 98    | 3333333333 |
| 004     | DDD  | 67    | 4444444444 |

**Tuple or record or row** : The single entry in the table is called tuple. The tuple represents a set of related data. In above **Student** table there are four tuples. One of the tuple can be represented as

|     |     |    |            |
|-----|-----|----|------------|
| 001 | AAA | 88 | 1111111111 |
|-----|-----|----|------------|

**Attribute or columns** : It is a part of table that contains several records. Each record can be broken down into several small parts of data known as attributes. For example the above table consists of four attributes such as **RollNo,Name,Marks** and **Phone**.

**Relation schema** : A relation schema describes the structure of the relation, with the name of the relation (i.e. name of table), its attributes and their names and type.

**Relation Instance** : It refers to specific instance of relation i.e. containing a specific set of rows. For example – the following is a relation instance – which contains the records with marks above 80.

| RollNo | Name | Marks | Phone      |
|--------|------|-------|------------|
| 001    | AAA  | 88    | 1111111111 |
| 002    | BBB  | 83    | 2222222222 |
| 003    | CCC  | 98    | 3333333333 |

**Domain** : For each attribute of relation, there is a set of permitted values called domain. For example – in above table, the domain of attribute **Marks** is set of all possible permitted marks of the students. Similarly the domain of **Name** attribute is all possible names of students.

That means Domain of Marks attribute is (88,83,98)

**Atomic :** The domain is atomic if elements of the domain are considered to be indivisible units. For example in above **Student** table, the attribute **Phone** is non-atomic.

**NULL attribute :** A null is a special symbol, independent of data type, which means either unknown or inapplicable. It does not mean zero or blank. For example - Consider a salary table that contains NULL

| Emp# | Job Name | Salary | Commission |
|------|----------|--------|------------|
| E10  | Sales    | 12500  | 32090      |
| E11  | Null     | 25000  | 8000       |
| E12  | Sales    | 44000  | 0          |
| E13  | Sales    | 44000  | Null       |

**Degree :** It is nothing but total number of columns present in the relational database. In given Student table –

| Roll No | Name | Marks | Phone      |
|---------|------|-------|------------|
| 001     | AAA  | 88    | 1111111111 |
| 002     | BBB  | 83    | 2222222222 |
| 003     | CCC  | 98    | 3333333333 |

The degree is 4.

**Cardinality :** It is total number of tuples present in the relational database. In above given table the cardinality is 3

**Example 4.2.1** Find out following for given Staff table

i) No of Columns

ii) No of tuples

iii) Different attributes

iv) Degree

v) Cardinality

| StaffID | Name  | Sex | Designation | Salary | DOJ          |
|---------|-------|-----|-------------|--------|--------------|
| S001    | John  | M   | Manager     | 50000  | 1 Oct. 2012  |
| S002    | Ram   | M   | Executive   | 20000  | 20 Jan. 2015 |
| S003    | Meena | F   | Supervisor  | 40000  | 12 Aug. 2011 |

**Solution :**

i) No of Columns = 6

ii) No of Tuples = 3

iii) Different attributes are StaffID, Name, Sex, Designation, Salary, DOJ

iv) Degree = Total number of columns = 6

v) Cardinality = Total number of rows = 3

**4.3 CODD's Rules****SPPU : Aug.-17, Oct.-19, Marks 5**

Codd proposed 13 rules for relational database management system, which are popularly known as **Codd's 12 rule** : These rules are as follows –

**Rule 0** : This rule states for a database to be relational, it must use its relational capabilities to manage the database.

**Rule 1 : The Information rule** - All information in an RDBMS is represented logically only by storing the values in tables.

**Rule 2 : The Guaranteed Access rule** - Each item of data in an RDBMS is guaranteed to be logically accessible by specifying the table name, primary key value, and column name.

**Rule 3 : The Systematic Treatment of Null Values rule** - Null values are supported in a fully relational DBMS for representing missing information or inapplicable information in a systematic way which is independent of the data type.

**Rule 4 : The Dynamic Online Catalog Based on the Relational Model rule** - Database dictionary which is called as catalog-is the structure description of the complete Database and it must be stored online. This Catalog must be governed by same rules as rest of the database. The same query language should be used on catalog as used to query database.

**Rule 5 : The Comprehensive Data Sublanguage rule** - At least one well structured, well-defined language must be there which can access all the data present in the database.

**Rule 6 : The View Updating rule** - All views of the data which are theoretically updatable must be updatable in practice by the DBMS.

**Rule 7 : Relational level operation** - The High-level Insert, Update, and Delete rule: There must be insert, delete and update operations at each level of relations.

**Rule 8 : The Physical Data Independence rule** - Physical storage should not matter the system. Whenever any changes are made in either storage representations or access methods then it should not affect the application.

**Rule 9 : The Logical Data Independence rule** - If any changes are made in table structure then the logical view of the user should not get affected. For Rule example - if a table is split into two tables internally, the view of the table to the user should be an entire table and not the split tables.

**Rule 10 : The Integrity Independence rule** - The Integrity constraints must be defined by the RDBMS stored in the system and it should not be enforced by the external application programs.

**Rule 11 : The Distribution Independence rule** - An RDBMS must have distribution independence. That means, even if database is scattered geographically, user should get a feel as if it is stored in one piece at one location.

**Rule 12 : The Non-sub-version rule** - If low-level language is allowed to access the system, then that low-level language must not be able to subvert or bypass the integrity rules which are expressed in a higher-level language.

### Review Questions

1. One of the rule designed by Codd's for good relational database management system is integrity independence, which states that all integrity constraints can be independently modified without the need of any change in the application. Justify the significance of rule in relational database management system. **SPPU : Aug.-17, In Sem, Marks 5**
2. Twelve rules are proposed by Codd, which according to him, a database must obey in order to be regarded as a true relational database. One of the rule is comprehensive data sub language rule. A database can only be accessed using a language having linear syntax that supports data definition, data manipulation and transaction management operations. Explain in brief above rule. Also state its significance. **SPPU : Oct.-19, In Sem, Marks 5**

## Part II : Relational Integrity

### 4.4 Keys

Keys are used to specify the tuples distinctly in the given relation.

Various types of keys used in relational model are – Superkey, Candidate Keys, primary keys, foreign keys. Let us discuss them with suitable example

#### 1) Super Key(SK) :

It is a set of one or more attributes within a table that can uniquely identify each record within a table. For example – Consider the Student table as follows –

| Reg No. | Roll No | Phone      | Name | Marks |
|---------|---------|------------|------|-------|
| R101    | 001     | 1111111111 | AAA  | 88    |
| R102    | 002     | 2222222222 | BBB  | 83    |
| R103    | 003     | 3333333333 | CCC  | 98    |
| R104    | 004     | 4444444444 | DDD  | 67    |

**Fig. 4.4.1 Student**

The superkey can be represented as follows

| Superkey |  | (RollNo, Phone, Name)<br>Superkey |           | (Name, Marks)<br>Not a Superkey |       |
|----------|--|-----------------------------------|-----------|---------------------------------|-------|
| RegNo.   |  | RollNo                            | Phone     | Name                            | Marks |
| R101     |  | 001                               | 111111111 | AAA                             | 88    |
| R102     |  | 002                               | 222222222 | BBB                             | 83    |
| R103     |  | 003                               | 333333333 | CCC                             | 98    |
| R104     |  | 004                               | 444444444 | DDD                             | 67    |

Clearly using the (RegNo) and (RollNo,Phone,Name) we can identify the records uniquely but (Name, Marks) of two students can be same, hence this combination not necessarily help in identifying the record uniquely.

## 2) Candidate Key(CK) :

The candidate key is a subset of superset. In other words candidate key is a single attribute or least or minimal combination of attributes that uniquely identify each record in the table. For example - in above given **Student** table, the candidate key is RegNo, (RollNo,Phone). The candidate key can be

| Candidate key |  | Candidate key |           |      |       |
|---------------|--|---------------|-----------|------|-------|
| RegNo.        |  | RollNo        | Phone     | Name | Marks |
| R101          |  | 001           | 111111111 | AAA  | 88    |
| R102          |  | 002           | 222222222 | BBB  | 83    |
| R103          |  | 003           | 333333333 | CCC  | 98    |
| R104          |  | 004           | 444444444 | DDD  | 67    |

Thus every candidate key is a superkey but every superkey is not a candidate key.

## 3) Primary Key(PK) :

The primary key is a candidate key chosen by the database designer to identify the tuple in the relation uniquely. For example – Consider the following representation of primary key in the student table



Primary key

| RegNo. | RollNo | Phone     | Name | Marks |
|--------|--------|-----------|------|-------|
| R101   | 001    | 111111111 | AAA  | 88    |
| R102   | 002    | 222222222 | BBB  | 83    |
| R103   | 003    | 333333333 | CCC  | 98    |
| R104   | 004    | 444444444 | DDD  | 67    |

Other than the above mentioned primary key, various possible primary keys can be **(RollNo)**, **(RollNo,Name)**, **(RollNo, Phone)**

The relation among super key, candidate key and primary can be denoted by

$$\text{Candidate Key} = \text{Super Key} - \text{Primary Key}$$

#### Rules for Primary Key

- i) The primary key may have one or more attributes.
- ii) There is **only one primary key** in the relation.
- iii) The value of primary key attribute can not be NULL.
- iv) The value of primary key attribute does not get changed.

#### 4) Alternate key :

The alternate key is a candidate key which is not chosen by the database designer to uniquely identify the tuples. For example –

Primary key                      Alternate key

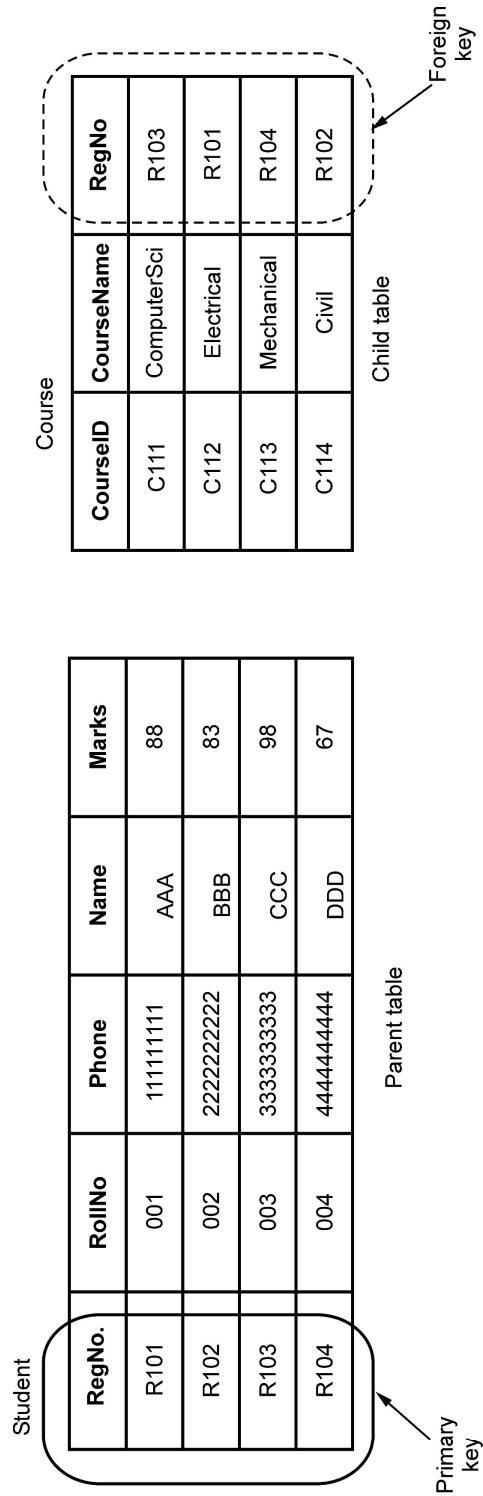
| RegNo. | RollNo | Phone     | Name | Marks |
|--------|--------|-----------|------|-------|
| R101   | 001    | 111111111 | AAA  | 88    |
| R102   | 002    | 222222222 | BBB  | 83    |
| R103   | 003    | 333333333 | CCC  | 98    |
| R104   | 004    | 444444444 | DDD  | 67    |

#### 5) Foreign key :

Foreign key is a single attribute or collection of attributes in one table that refers to the primary key of other table.

- Thus foreign keys refer to primary key.
- The table containing the primary key is called **parent table** and the table containing foreign key is called **child table**.

- Example -



From above example, we can see that two tables are linked. For instance we could easily find out that the 'Student CCC has opted for ComputerSci course'

## 4.5 Constraints

**SPPU : May-18, Dec.-19, Marks 5**

Constraints mean some rules or restrictions that are set on the database.

There are three main types of constraints.

1. Domain Constraint
2. Key Constraint or NULL Constraint
3. Integrity Constraint
  - i) Entity Integrity Constraint
  - ii) Referential Integrity Constraint

### 1. Domain Constraint

- Domain constraint defines the domain or set of values for an attribute.
- The data type of domain includes string, character, integer, time, date, currency, etc. The value of the attribute must be available in the corresponding domain.
- For example - Consider the Student table as follows.

| Roll No | Name | Marks | Phone     |
|---------|------|-------|-----------|
| 001     | AAA  | 88    | 111111111 |
| 001     | BBB  | 83    | 222222222 |
| 003     | 1234 | 98    | 333333333 |
| 004     | DDD  | 67    | 444444444 |

Name cannot be numeric value.  
It must be a string.  
Hence this is not allowed !!!

The above relation does not satisfy the domain constraint.

### 2. Key Constraint or Null Constraint

- Keys are used to identify particular record from the table. Primary key is normally used to identify the record uniquely.

- Hence the key constraint can be stated as -
  - All values of primary key must be unique.
  - The value of primary key must not be NULL.
- For example - Consider the Student table as follows. For this relation, the **Roll No** is a primary key. It is expected to find the desired record using this primary key.

| Roll No | Name | Marks | Phone     |
|---------|------|-------|-----------|
| 001     | AAA  | 88    | 111111111 |
| 001     | BBB  | 83    | 222222222 |
| 003     | CCC  | 98    | 333333333 |
| 004     | DDD  | 67    | 444444444 |

Duplicate values ?  
This is not allowed !!!

The above relation does not satisfy key constraint as the primary key is not having unique value.

### 3. Integrity Constraint

- Integrity constraints are rules that are to be applied on database columns to ensure the validity of data.
- For example -
  - i) The Employee ID and Department ID must consist of two digits.
  - ii) Every Employee ID must start with letter.

#### i) Entity Integrity Constraint

- **This rule states that** “In the relations, the value of attribute of primary key can not be null”.
- The NULL represents a value for an attribute that is currently unknown or is not applicable for this tuple. The Nulls are always **to deal with incomplete or exceptional data**.
- The primary key value helps in uniquely identifying every row in the table. Thus if the users of the database want to retrieve any row from the table or perform any action on that table, they must know the value of the key for that row. Hence it is necessary that the primary key should not have the NULL value.

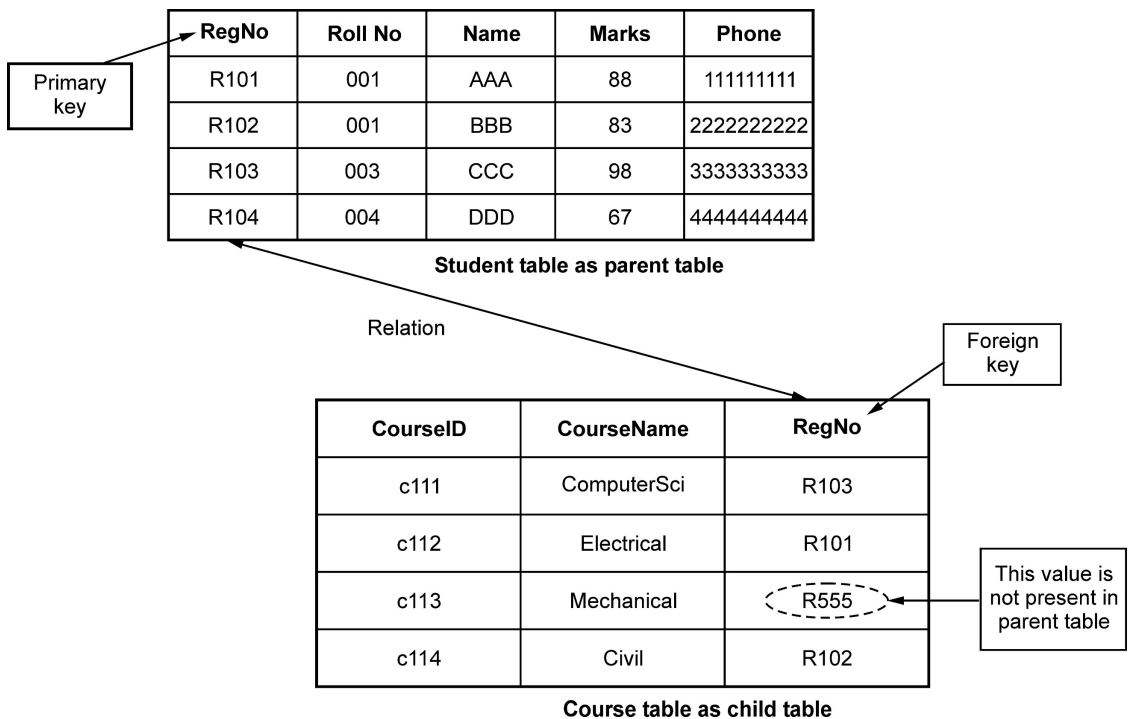
- For example -

| Roll No | Name | Marks | Phone     |
|---------|------|-------|-----------|
| 001     | AAA  | 88    | 111111111 |
| 001     | BBB  | 83    | 222222222 |
| 003     | CCC  | 98    | 333333333 |
|         | DDD  | 67    | 444444444 |

NULL is not allowed !!!

## ii) Referential Integrity Constraint

- In relationships, data is linked between two or more tables.
- This is achieved by having the foreign key (in the associated table) reference a primary key value (in the primary - or parent - table). Because of this, we need to ensure that data on both sides of the relationship remain intact.
- **The referential integrity rule states that “whenever a foreign key value is used it must reference a valid, existing primary key in the parent table”.**
- For example - Consider two tables



In above relation, the registration no. R555 is not existing still if it is present in the course table, then we say that it is not following referential integrity constraint.

### Review Question

1. Explain the concepts of referential integrity constraint and entity integrity constraint with example.

**SPPU : May-18, Dec.-19, End Sem, Marks 5**

## 4.6 Enterprise Constraints

Enterprise constraints are also called as semantic constraints.

The enterprise constraints are basically the additional rules specified by users or database administrators. These constraints are normally based on multiple tables.

Examples of enterprise constraints are –

- 1) The salary of teacher should not exceed the salary of Principal.
- 2) A Student can not opt for more than two courses at a time.
- 3) A class can have maximum 50 students.

## Part III : Database Design

## 4.7 Features of Good Relational Designs

**SPPU : Dec.-18, Aug.-17, Marks 5**

There are two primary goals of relational database design –

- i) To generate a set of relation schemas that allows us to store information without unnecessary redundancy and
- ii) To allow us to retrieve information easily.

**Example 4.7.1** Explain what is meant by repetition of information and inability to represent information. Explain why each of these properties may indicate a bad relational database design.

**SPPU ; Dec.-18, End Sem, Marks 5**

**Solution :** Repetition of information and inability to represent the required information are considered to be bad features of relational design. Consider following schema

| EmpID | EName | Salary | DeptID | DeptName | DeptLoc |
|-------|-------|--------|--------|----------|---------|
| 1     | AAA   | 10000  | 101    | XYZ      | Pune    |
| 2     | BBB   | 20000  | 101    | XYZ      | Pune    |
| 3     | CCC   | 30000  | 101    | XYZ      | Pune    |
| 4     | DDD   | 40000  | 102    | PQR      | Mumbai  |

Now if we want to insert a record 5,EEE,50000 for the DeptID 101, then again there will be repetition of information about DeptID, DeptName and DeptLoc i.e. (101,XYZ,Pune). That means if we want to perform some operation (insertion, updation, deletion) on the relational schema then it should not cause repetition of information.

The above scenario indicates bad database design.

Inability to represent information is a condition where a relationship exists among only a proper subset of the attributes in a relation. This is bad relational database design because all the unrelated attributes must be filled with null values otherwise a tuple without the unrelated information cannot be inserted into the relation.

### Review Question

1. Explain different features of good relational database design.

**SPPU : Aug.-17, In Sem, Marks 5**

## 4.8 Data Redundancy and Update Anomalies

**Definition :** Data redundancy is a condition created in database in which same piece of data is held at two different places.

**Redundancy** is at the root of several problems associated with relational schemas.

**Problems caused by redundancy :** Following problems can be caused by redundancy-

- i) **Redundant storage :** Some information is stored repeatedly.
- ii) **Update anomalies :** If one copy of such repeated data is updated then inconsistency is created unless all other copies are similarly updated.
- iii) **Insertion anomalies :** Due to insertion of new record repeated information get added to the relation schema.
- iv) **Deletion anomalies :** Due to deletion of particular record some other important information associated with the deleted record get deleted and thus we may lose some other important information from the schema.

**Example :** Following example illustrates the above discussed anomalies or redundancy problems.

Consider following Schema in which all possible information about Employee is stored.

| EmpID | EName | Salary | DeptID | DeptName | DeptLoc |
|-------|-------|--------|--------|----------|---------|
| 1     | AAA   | 10000  | 101    | XYZ      | Pune    |
| 2     | BBB   | 20000  | 101    | XYZ      | Pune    |
| 3     | CCC   | 30000  | 101    | XYZ      | Pune    |
| 4     | DDD   | 40000  | 102    | PQR      | Mumbai  |

Redundancy!!!

- 1) **Redundant storage** : Note that the information about DeptID, DeptName and DeptLoc is repeated.
- 2) **Update anomalies** : In above table if we change **DeptLoc** of Pune to Chennai, then it will result **inconsistency** as for DeptID 101 the DeptLoc is Pune. Or otherwise, we need to **update multiple copies** of **DeptLoc** from Pune to Chennai. Hence this is an update anomaly.
- 3) **Insertion anomalies** : For above table if we want to add new tuple say (5, EEE, 50000) for **DeptID 101** then it will cause repeated information of (101, XYZ, Pune) will occur.
- 4) **Deletion anomalies** : For above table, if we delete a record for **EmpID 4**, then automatically information about the **DeptID 102, DeptName PQR** and **DeptLoc Mumbai** will get deleted and one may not be aware about **DeptID 102**. This causes deletion anomaly.

## 4.9 Normalization

SPPU : Dec.-17, Marks 5

- Normalization is the process of reorganizing data in a database so that it meets **two basic requirements** :
  - 1) There is **no redundancy** of data (all data is stored in only one place) and
  - 2) **Data dependencies** are logical (all related data items are stored together)

### Need for normalization

- 1) **It eliminates redundant data.**
- 2) **It reduces chances of data error.**
- 3) The normalization is important because it allows database to take up **less disk space**.



- 4) It also help in increasing the **performance**.
- 5) It improves the data integrity and consistency.

### Review Question

1. What is normalization ? What is the need of normalized database ?

**SPPU : Dec.-17, End Sem, Marks 5**

### 4.10 Atomic Domains and First Normal Form

By atomic value, we mean that each value in the domain is indivisible.

The first normal form rule defines that all the attributes in a relation must have atomic domains. The values in an atomic domain are indivisible units.

The table is said to be in 1NF if it follows following rules -

- i) It should only have single (atomic) valued attributes/columns.
- ii) Values stored in a column should be of the same domain.
- iii) All the columns in a table should have unique names.
- iv) And the order in which data is stored, does not matter.

Consider following student table

**Student**

| sid | sname | Phone |
|-----|-------|-------|
| 1   | AAA   | 11111 |
|     |       | 22222 |
| 2   | BBB   | 33333 |
| 3   | CCC   | 44444 |
|     |       | 55555 |

As there are multiple values of phone number for sid 1 and 3, the above table is not in 1NF. We can make it in 1NF. The conversion is as follows -

| sid | sname | Phone |
|-----|-------|-------|
| 1   | AAA   | 11111 |
| 1   | AAA   | 22222 |
| 2   | BBB   | 33333 |
| 3   | CCC   | 44444 |
| 3   | CCC   | 55555 |

**Review Question**

1. Suggest and explain three different techniques to achieve 1NF using suitable example.

**4.11 Decomposition using Functional Dependencies****SPPU : Oct.-19, May-18, Marks 5**

**Definition :** A functional dependency  $A \rightarrow B$  in a relation holds if two tuples having same value of attribute A also have the same value for attribute B. It is denoted by  $A \rightarrow B$  where A is called determinant and B is called dependent.

For example - Consider Student table as follows -

| Roll | Name | City        |
|------|------|-------------|
| 1    | AAA  | Mumbai      |
| 2    | BBB  | Pune        |
| 3    | CCC  | Gandhinagar |

Here

Roll  $\rightarrow$  Name **hold**

But

Name  $\rightarrow$  City **does not hold**

- In above table, student roll number is unique hence each student's name and city can be uniquely identified using his roll number.
- But using name we cannot uniquely identify his/her city because there can be same names of the students. Similarly using city name we can not identify the student uniquely. As in the same city may belong to multiple students.

**Another example -**

Consider a relation in which the roll of the student and his/her name is stored as follows :

| R | N   |
|---|-----|
| 1 | AAA |
| 2 | BBB |
| 3 | CCC |
| 4 | DDD |
| 5 | EEE |

**Fig. 4.11.1 : Table which holds functional dependency i.e.  $R \rightarrow N$**

Here,  $R \rightarrow N$  is true. That means the functional dependency holds true here. Because for every assigned RollNumber of student there will be unique name. For instance : The name of the Student whose RollNo is 1 is AAA. But if we get two different names for the same roll number then that means the table does not hold the functional dependency. Following is such table -

| R | N   |
|---|-----|
| 1 | AAA |
| 2 | BBB |
| 3 | CCC |
| 1 | XXX |
| 2 | YYY |

**Fig. 4.11.2 : Table which does not hold functional dependency**

In above table for RollNumber 1 we are getting two different names - "AAA" and "XXX". Hence here it does not hold the functional dependency.

**Trivial FD :** The functional dependency  $A \rightarrow B$  is trivial if B is a subset of A.

For example  $(A,B) \rightarrow A$

**Non Trivial FD :** The functional dependency  $A \rightarrow B$  is non trivial if B is not a subset of A.

For example  $\{A,B\} \rightarrow C$

**Example 4.11.1** For the given below relation  $R(A,B,C,D,E)$  and its instance, check whether FDs given hold or not. Give reasons.

i)  $A \rightarrow B$  ii)  $B \rightarrow C$  iii)  $D \rightarrow E$  iv)  $CD \rightarrow E$

| A  | B  | C  | D  | E  |
|----|----|----|----|----|
| a1 | b1 | c1 | d1 | e1 |
| a1 | b2 | c1 | d1 | e1 |
| a2 | b2 | c1 | d2 | e3 |
| a2 | b3 | c3 | d2 | e2 |

**Solution :** Association among attributes is known as Functional Dependencies (FD). AFD  $X \rightarrow Y$  require that the value of X uniquely determines the value of Y where X and Y are set of attributes.

For example,

Roll\_No  $\rightarrow$  Name : the value of Roll\_No uniquely determines the Name.

Now from, the given relation and its instance -

- i) The FD  $A \rightarrow B$  does not hold because – a1 has two different values b1 and b2. Similarly a2 has two different values and those are b2 and b3.
- ii) The FD  $B \rightarrow C$  holds true.
- iii)  $D \rightarrow E$  does not hold true because d2 gives two different values e3 and e2.
- iv)  $CD \rightarrow E$  hold true as (c1,d1) gives e1 , (c1,d2) gives e3 and (c3,d2) gives e2. All are uniquely identified.

#### 4.11.1 Inference Rules

The closure set is a set of all functional dependencies implied by a given set F. It is denoted by  $F^+$

The closure set of functional dependency can be computed using basic three rules which are also called as **Armstrong's Axioms**.

These are as follows -

- i) **Reflexivity** : If  $X \supseteq Y$ , then  $X \rightarrow Y$
- ii) **Augmentation** : If  $X \rightarrow Y$ , then  $XZ \rightarrow YZ$  for any Z
- iii) **Transitivity** : If  $X \rightarrow Y$  and  $Y \rightarrow Z$ , then  $X \rightarrow Z$

In addition to above axioms some additional rules for computing closure set of functional dependency are as follows -

- **Union** : If  $X \rightarrow Y$  and  $X \rightarrow Z$  then  $X \rightarrow YZ$
- **Decomposition** : If  $X \rightarrow YZ$ , then  $X \rightarrow Y$  and  $X \rightarrow Z$

Let us understand how to apply Armstrong's axioms for finding the closure of set of functional dependencies -

**Example 4.11.2** Given FD's for relation  $R\{A,B,C,D,E,F\}$ , Find closure of FD set by applying Armstrong's Axioms.

$A \rightarrow B, A \rightarrow C, CD \rightarrow E, CD \rightarrow F, B \rightarrow E$

**Solution :**

**Step 1 :**  $A \rightarrow$  gives A attribute itself by reflexivity. It is called trivial production.

$A \rightarrow B$  and  $A \rightarrow C$ , Hence by union rule  $A \rightarrow BC$

$A \rightarrow B$  and  $B \rightarrow E$ , Hence by transitivity rule  $A \rightarrow E$

$\therefore (A)^+ = \{A, B, C, E\}$

**Step 2 :**  $B \rightarrow$  gives B itself

And  $B \rightarrow E$

$\therefore (B)^+ = \{B, E\}$

**Step 3 :**  $CD \rightarrow$  gives CD itself. It is trivial.

$CD \rightarrow E$ ,  $CD \rightarrow F$ , Hence by union rule  $CD \rightarrow EF$

$\therefore (CD)^+ = \{C, D, E, F\}$

So by omitting trivial productions, we get

$\therefore F^+ = \{A \rightarrow BC, A \rightarrow E, B \rightarrow E, CD \rightarrow EF\}$

**Example 4.11.3** Compute the closure of the following set  $F$  of functional dependencies for relational schema  $R = (A, B, C, D, E)$   $A \rightarrow BC$ ,  $CD \rightarrow E$ ,  $B \rightarrow D$ ,  $E \rightarrow A$

**Solution :** The closure of  $F$  is denoted by  $F^+$  and it can be computed in following steps

**Step 1 :** As  $A \rightarrow BC$  is given we get

$A \rightarrow B$  and  $A \rightarrow C$

By decomposition rule

**Step 2 :** As

$A \rightarrow B$

(Refer step 1)

$B \rightarrow D$

(given)

$A \rightarrow D$

(transitivity rule)

**Step 3 :**

$A \rightarrow CD$  because  $A \rightarrow C$  and  $A \rightarrow D$

(From step 1 and Step 2 applying union rule)

**Step 4 :**

$CD \rightarrow E$

(given)

$A \rightarrow E$

(transitive rule as  $A \rightarrow CD$ ,  $CD \rightarrow E$   
hence  $A \rightarrow E$ )

**Step 5 :**

Since  $A \rightarrow A$ ,  
 $A \rightarrow ABCDE$

we have (reflexive)  
 from the above steps (union)

**Step 6 :**

Since  $E \rightarrow A$ ,  $E \rightarrow ABCDE$

(transitive)

**Step 7 :**

Since  $CD \rightarrow E$ ,  $CD \rightarrow ABCDE$

(transitive)

**Step 8 :**

Since  $B \rightarrow D$  and  $BC \rightarrow CD$ ,  $BC \rightarrow ABCDE$  (augmentative, transitive)

**Step 9 :**

Also,  $C \rightarrow C$ ,  $D \rightarrow D$ ,  $BD \rightarrow D$

Thus any functional dependency with  $A$ ,  $E$ ,  $BC$ , or  $CD$  on the left hand side of the arrow is in  $F^+$

**Example 4.11.4** Give Armstrong's axioms and using it find the closure of following FD set.

$A \rightarrow B$ ,  $AB \rightarrow C$ ,  $D \rightarrow AC$ ,  $D \rightarrow E$

**Solution :**

**Step 1 :** Consider the rules  $D \rightarrow AC$  and  $D \rightarrow E$

$\therefore D \rightarrow ACE$

(Union rule)

**Step 2 :** Consider  $AB \rightarrow C$  then we get

$\therefore A \rightarrow C$  and  $B \rightarrow C$

(decomposition rule)

Thus  $F^+ = \{A \rightarrow C, B \rightarrow C, D \rightarrow ACE\}$

**Example 4.11.5**  $R = \{A, B, C, D, E, F\}$  and FDs are  $A \rightarrow BC$   $E \rightarrow CF$   $B \rightarrow E$   $CD \rightarrow EF$  compute closure of  $\{A, B\}^+$

**Solution :**

**Step 1 :**  $A \rightarrow BC$

$\therefore A \rightarrow B$  and  $A \rightarrow C$

(decomposition)

So we add  $A$ ,  $B$ ,  $C$  in closure set

**Step 2 : E → CF**

∴ E → C and E → F (decomposition)

**Step 3 : B → E ∴ B → C, F**

So we add E and F in closure set

Hence

$$\{A, B\}^+ = \{A, B, C, E, F\}$$

**Example 4.11.6** Consider schema EMPLOYEE(E-ID, E-NAME, E-CITY, E-STATE) and

FD = {E-ID → E-NAME, E-ID → E-CITY, E-ID → E-STATE, E-CITY → E-STATE}

(1) Find attribute of closure for (E-ID)<sup>+</sup>

(2) Find (E-NAME)<sup>+</sup>

**Solution :**

- 1) Finding (E-ID)<sup>+</sup> means finding the closure. In this process we try to find out, all the attributes that can be derived from E-ID.

As E-ID → E-NAME, E-ID → E-CITY, E-ID → E-STATE, We add E-NAME, E-ID, E-CITY, E-STATE to (E-ID)<sup>+</sup>

As E-ID → E-CITY, E-CITY → E-STATE, we add E-STATE to (E-ID)<sup>+</sup>

$$\therefore (E-ID)^+ = \{E-ID, E-NAME, E-CITY, E-STATE\}$$

- 2) E-NAME derives no rule. Hence (E-NAME)<sup>+</sup> = {E-NAME}

**4.11.2 Keys and Functional Dependencies**

- For a given relation R = {A1, A2, A3, ..., An} K is a key of R then if closure (K)<sup>+</sup> = {A1, A2, ..., An} and no subset of K i.e. X such that (X)<sup>+</sup> = A1, A2, ..., An
- In other words there are two conditions -
  - 1) The (K)<sup>+</sup> contains all the attributes of relations R -
  - 2) All subset X of K, (X)<sup>+</sup> never contains all the attributes of R i.e. (X)<sup>+</sup> ≠ {A1, A2, ..., An}
- If only one subset of R satisfy above condition then it is known as **primary key**.
- If more than one subset of R satisfies above condition then all these subsets are recognized as **candidate keys**. In that case one of the candidate key is also considered as **primary key**.
- A superset of candidate key K is known as **superkey**.

**Example 4.11.7** Give  $R = \{A, B, C, G, H, I\}$ . The following set  $F$  of functional dependencies holds

$A \rightarrow B, A \rightarrow C, CG \rightarrow H, CG \rightarrow I, B \rightarrow H$

Compute  $AG^+$ . Is  $AG$  candidate key?

**Solution :**

**Step 1 :**  $A \rightarrow B, A \rightarrow C$ , Hence add  $A, B, C$  to the set of  $AG^+$ .

**Step 2 :**  $A \rightarrow B, B \rightarrow H$ , hence add  $H$  to the set of  $AG^+$

**Step 3 :**  $CG \rightarrow I$ , hence add  $I$  to the set of  $AG^+$ . Also add  $G$  to the set.

Thus  $(AG)^+ = \{A, B, C, G, H, I\} = \text{Relation } R$

Hence  $(AG)^+$  is a candidate key.

**Example 4.11.8** Compute the closure of  $R(A, B, C, D, E)$  with the following set of functional dependencies  $A \rightarrow BC, CD \rightarrow E, B \rightarrow D, E \rightarrow A$

List the candidate keys of  $R$

**Solution :**

**Step 1 :**  $A \rightarrow BC$  hence add  $A, B, C$  to  $(A)^+$

$A \rightarrow BC$  can be decomposed into  $A \rightarrow B$  and  $A \rightarrow C$ . Also  $B \rightarrow D$ . Thus  $A \rightarrow D$  is also true by transitivity rule.

Hence add  $D$  to  $(A)^+$

$A \rightarrow C, A \rightarrow D \therefore$  By union rule  $A \rightarrow CD$ .

As  $CD \rightarrow E$  add  $E$  to  $(A)^+$

$\therefore (A)^+ = \{A, B, C, D, E\}$

**Step 2 :** Consider  $(B)^+ = \{B, D\} \neq R$  hence it is not a candidate key

**Step 3 :** Consider  $(BC)^+ = \{B, C, D, E, A\} = \{A, B, C, D, E\} = R$ . Hence it is a candidate key

**Step 4 :** Consider  $CD \rightarrow E, E \rightarrow A$ , hence  $(CD)^+ = \{A, B, C, D, E\}$ .

Hence it is a candidate key

**Step 5 :** Consider  $E \rightarrow A, A \rightarrow BC, B \rightarrow D, CD \rightarrow E$ .

Hence  $(E)^+ = \{A, B, C, D, E\}$  is a candidate key.

Thus we get the candidate keys as  $\{A, BC, CD, E\}$



**Example 4.11.9** Consider schema  $R = (A, B, C, G, H, I)$  and the set  $F$  of functional dependencies  $\{A \rightarrow B, A \rightarrow C, CG \rightarrow H, CG \rightarrow I, B \rightarrow H\}$ . Use  $(F)^+$  Prove  $(AG)^+ \rightarrow I$

**Solution :**

**Step 1 :** As  $A \rightarrow B$   
 $B \rightarrow H$   
 $\therefore A \rightarrow H$  (Transitivity rule)

**Step 2 :**  $CG \rightarrow H$   
 $CG \rightarrow I$   
 $\therefore CG \rightarrow HI$  (Union rule)

**Step 3 :**  $A \rightarrow C$   
 $CG \rightarrow I$   
 $AG \rightarrow I$  (Pseudo transitive rule)

Thus  $AG \rightarrow I$  is proved

$(F)^+ = \{A \rightarrow B, A \rightarrow C, CG \rightarrow H, CG \rightarrow I, B \rightarrow H, A \rightarrow H, CG \rightarrow HI, AG \rightarrow I\}$

### Review Questions

1. Explain in brief with suitable example full functional dependency and partial dependency.

**SPPU : Oct.-19, In Sem, Marks 3**

2. What is the impact of insert, update and delete anomaly on overall design of database ? How normalization is used to remove these anomalies ?

**SPPU : May-18, End Sem, Marks 5**

### 4.12 Equivalence and Minimal Cover

- A set of functional dependencies  $E$  is said to be covered by  $F$  if every FD in  $E$  is also in  $F^+$ , i.e. every dependency in  $F$  can be inferred from  $E$  and vice versa.
- Two sets of FDs  $E$  and  $F$  are equivalent if  $E^+ = F^+$ .
- If  $E$  and  $F$  are equivalent only if both  $E^+ = F^+$ 
  - $E$  covers  $F$
  - $F$  covers  $E$  holds

**Example 4.12.1** Here are two sets of FDs for  $R(A, B, C, D, E)$ . Are they equivalent ?

- $A \rightarrow B$
- $A \rightarrow BC$
- $AB \rightarrow C$
- $D \rightarrow AE$
- $D \rightarrow AC$
- $D \rightarrow E$

**Solution :** Two sets of FDs are said to be equivalent if

**Rule 1 :** If  $FD2 \supset FD1$ . That means all FDs of  $FD1$  can be derived from all the FDs of  $FD2$ .

**Rule 2 :** If  $FD1 \supset FD2$ . That means all FDs of  $FD2$  can be derived from all the FDs of  $FD1$ .

**Rule 3 :** If both rule 1 and rule 2 are true then  $FD1 = FD2$ .

For given two sets

**Step 1 :** We will first check if all the FDs of  $FD1$  are present in  $FD2$

|                                                                                                                                                      |                                                                                                                   |
|------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| A $\rightarrow$ B is present in $FD1$ .                                                                                                              | A $\rightarrow$ BC is in $FD2$ , that also means<br>A $\rightarrow$ B and A $\rightarrow$ C by decomposition rule |
| Similarly AB $\rightarrow$ C is in $FD1$<br>$\therefore (A)^+ = \{A, B, C\}$                                                                         | Hence<br>$(A)^+ = \{A, B, C\}$                                                                                    |
| $\therefore (AB)^+ = \{A, B, C\}$                                                                                                                    | As D $\rightarrow$ AE then by decomposition rule,                                                                 |
| D $\rightarrow$ AC                                                                                                                                   | D $\rightarrow$ A, D $\rightarrow$ E                                                                              |
| i.e D $\rightarrow$ A, D $\rightarrow$ C by decomposition rule.                                                                                      | As A $\rightarrow$ B, then by transitivity rule D $\rightarrow$ A,<br>A $\rightarrow$ B, D $\rightarrow$ B        |
| The D $\rightarrow$ A, A $\rightarrow$ B and C                                                                                                       | $\therefore (D)^+ = \{A, B, C, D, E\}$                                                                            |
| D $\rightarrow$ A, D $\rightarrow$ B, D $\rightarrow$ C by transitivity rule<br>D $\rightarrow$ E is given<br>$\therefore (D)^+ = \{A, B, C, D, E\}$ |                                                                                                                   |

**Step 2 :** We will first check if all the FDs of  $FD2$  are present in  $FD1$

|                                        |                                        |
|----------------------------------------|----------------------------------------|
| $\therefore (A)^+ = \{A, B, C\}$       | $(A)^+ = \{A, B, C\}$                  |
| $\therefore (AC)^+ = \{A, B, C\}$      | $\therefore (D)^+ = \{A, B, C, D, E\}$ |
| $\therefore (D)^+ = \{A, B, C, D, E\}$ |                                        |

Thus from Step 1 and Step 2,  $FD2 \supset FD1$  and  $FD1 \supset FD2$ . Hence both the sets are equivalent.

**Example 4.12.2** Consider two sets of functional dependency.

$F = \{A \rightarrow C, AC \rightarrow D, E \rightarrow AD, E \rightarrow H\}$  and  $G = \{A \rightarrow CD, E \rightarrow AH\}$ . Are they equivalent ?

**Solution :** We will find the FDs of both F and G and then check for  $F \supset G$  and  $G \supset F$ .

**Step 1 : For F**

Using G functional dependencies -

$$(A)^+ = \{A, C, D\} \leftarrow \text{As } A \rightarrow CD \text{ is in G}$$

$$(AC)^+ = \{A, C, D\} \leftarrow \text{As } A \rightarrow CD \text{ is in G}$$

$$(E)^+ = \{A, C, D, E, H\} \leftarrow \text{As } E \rightarrow AH, A \rightarrow CD \text{ is in G}$$

Using F functional dependencies -

$$(A)^+ = \{A, C, D\} \leftarrow \text{As } A \rightarrow C \text{ and } AC \rightarrow D \text{ is in F}$$

$$(AC)^+ = \{A, C, D\} \leftarrow AC \rightarrow D \text{ is in F}$$

$$(E)^+ = \{A, C, D, E, H\} \leftarrow \text{As } E \rightarrow AD, E \rightarrow H \text{ and } A \rightarrow C \text{ is in F}$$

**Step 2 : For G**

Using F functional dependencies -

$$(A)^+ = \{A, C, D\} \leftarrow \text{As } A \rightarrow C \text{ and } AC \rightarrow D \text{ is in F}$$

$$(E)^+ = \{A, C, D, E, H\} \leftarrow \text{As } E \rightarrow AD, E \rightarrow H \text{ is in F}$$

Using G functional dependencies -

$$(A)^+ = \{A, C, D\} \leftarrow \text{As } A \rightarrow CD \text{ is in G}$$

$$(E)^+ = \{A, C, D, E, H\} \leftarrow \text{As } E \rightarrow AH \text{ and } A \rightarrow CD \text{ is in G}$$

**Step 3 :** From both these steps

**$G \supset F$  and  $F \supset G$**

Hence F and G are equivalent.

**Example 4.12.3** Given below are two sets of FDs for a relation  $R(A, B, C, D, E)$ , Are they equivalent ?

i)  $A \rightarrow B, AB \rightarrow C, D \rightarrow AC, D \rightarrow E$  ii)  $A \rightarrow BC, D \rightarrow AE$

**Solution :** We will assume these relations as F and G. That means -

**F :**  $A \rightarrow B, AB \rightarrow C, D \rightarrow AC, D \rightarrow E$

**G :**  $A \rightarrow BC, D \rightarrow AE$

Now we will find the  $F^+$  and  $G^+$  as follows

**Step 1 : For F**

Using G functional dependencies -

$$(A)^+ = \{A, B, C\}$$

$$(AB)^+ = \{A, B, C\}$$

$$(D)^+ = \{D, A, C, E, B\}$$

Using F functional dependencies -

$$(A)^+ = \{A, B, C\}$$

$$(AB)^+ = \{A, B, C\}$$

$$(D)^+ = \{D, A, C, E, B\}$$

**Step 2 : For G**

Using F functional dependencies -

$$(A)^+ = \{A, B, C\}$$

$$(D)^+ = \{D, A, C, E, B\}$$

Using G functional dependencies -

$$(A)^+ = \{A, B, C\}$$

$$(D)^+ = \{D, A, C, E, B\}$$

**Step 3 : From both these steps**

**$G \supseteq F$  and  $F \supseteq G$**

Hence F and G are equivalent.

**Minimal cover**

**Formal definition :** A minimal cover for a set F of FDs is a set G of FDs such that :

- 1) Every dependency in G is of the form  $X \rightarrow A$ , where A is a single attribute.
- 2) **The closure  $F^+$  is equal to the closure  $G^+$ .**
- 3) If we obtain a set H of dependencies from G by deleting one or more dependencies or by deleting attributes from a dependency in G, then  $F^+ \neq H^+$ .

**Concept of extraneous attributes**

**Definition :** An attribute of a functional dependency is said to be extraneous if we can remove it without changing the closure of the set of functional dependencies. The formal definition of extraneous attributes is as follows :

Consider a set  $F$  of functional dependencies and the functional dependency  $\alpha \rightarrow \beta$  in  $F$

- Attribute  $A$  is extraneous in  $\alpha$  if  $A \in \alpha$  and  $F$  logically implies  $(F - \{\alpha \rightarrow \beta\}) \cup \{(\alpha - A) \rightarrow \beta\}$
- Attribute  $A$  is extraneous in  $\beta$  if  $A \in \beta$  and the set of functional dependencies  $(F - \{\alpha \rightarrow \beta\}) \cup \{(\alpha \rightarrow (\beta - A))\}$  logically implies  $F$ .

### Algorithm for computing canonical cover for set of functional dependencies $F$

$F_C = F$

#### repeat

Use the union rule to replace any dependencies in  $F_C$  of the form

$\alpha_1 \rightarrow \beta_1$  and  $\alpha_1 \rightarrow \beta_2$  and  $\alpha_1 \rightarrow \beta_1 \beta_2$

Find a functional dependency  $\alpha \rightarrow \beta$  in  $F_C$  with an extraneous attribute either in  $\alpha$  or in  $\beta$ .

/\* The test for extraneous attributes is done using  $F_C$ , not  $F$  \*/

If an extraneous attribute is found, delete it from  $\alpha \rightarrow \beta$  in  $F_C$ .

until ( $F_C$  does not change)

**Example 4.12.4** Consider the following functional dependencies over the attribute set  $R(ABCDE)$  for finding minimal cover  $FD = \{A \rightarrow C, AC \rightarrow D, B \rightarrow ADE\}$ .

**Solution :**

**Step 1 : Split the FD** such that R.H.S contain single attribute. Hence we get

$A \rightarrow C$

$AC \rightarrow D$

$B \rightarrow A$

$B \rightarrow D$

$B \rightarrow E$

**Step 2 :** Find the **redundant entries** and **delete** them. This can be done as follows -

- **For  $A \rightarrow C$  :** We find  $(A)^+$  by assuming that we delete  $A \rightarrow C$  temporarily. We get  $(A)^+ = \{A\}$ . Thus from  $A$  it is not possible to obtain  $C$  by deleting  $A \rightarrow C$ . This means we can not delete  $A \rightarrow C$ .
- **For  $AC \rightarrow D$  :** We find  $(AC)^+$  by assuming that we delete  $AC \rightarrow D$  temporarily. We get  $(AC)^+ = \{AC\}$ . Thus by such deletion it is not possible to obtain  $D$ . This means we can not delete  $AC \rightarrow D$ .

- **For  $B \rightarrow A$  :** We find  $(B)^+$  by assuming that we delete  $B \rightarrow A$  temporarily. We get  $(B)^+ = \{BDE\}$ . Thus by such deletion it is not possible to obtain A. This means we can not delete  $B \rightarrow A$ .
- **For  $B \rightarrow D$  :** We find  $(B)^+$  by assuming that we delete  $B \rightarrow D$  temporarily. We get  $(B)^+ = \{BEACD\}$ . This shows clearly that even if we delete  $B \rightarrow D$  we can obtain D. This means we can delete  $B \rightarrow A$ . Thus it is redundant.
- **For  $B \rightarrow E$  :** We find  $(B)^+$  by assuming that we delete  $B \rightarrow E$  temporarily. We get  $(B)^+ = \{BDAC\}$ . Thus by such deletion it is not possible to obtain E. This means we can not delete  $B \rightarrow E$ .

To summarize we get now

$A \rightarrow C$

$AC \rightarrow D$

$B \rightarrow A$

$B \rightarrow E$

Thus R.H.S gets simplified.

**Step 3 :** Now we will simplify L.H.S.

Consider  $AC \rightarrow D$ . Here we can split A and C. For that we find closure set of A and C.

$$(A)^+ = (AC)$$

$$(C)^+ = (C)$$

Thus C can be obtained from both A as well as C. That also means we need not have to have AC on L.H.S. Instead, only A can be allowed and C can be eliminated. Thus after simplification we get

$A \rightarrow D$

To summarize we get now

$A \rightarrow C$

$A \rightarrow D$

$B \rightarrow A$

$B \rightarrow E$

Thus L.H.S gets simplified.

**Step 3 :** The simplified L.H.S. and R.H.S can be combined together to form

$A \rightarrow CD$

$B \rightarrow AE$

This is a **minimal cover** or **canonical cover** of functional dependencies.

**Example 4.12.5** A relation  $R(A,C,D,E,H)$  satisfies the following FDs  $A \rightarrow C$ ,  $AC \rightarrow D$ ,  $E \rightarrow AD$ ,  $E \rightarrow H$ . Find the canonical cover for this set of FD's.

**Solution :** For obtaining canonical cover we have to find the redundant entries from both LHS and RHS and eliminate them.

**Step 1 :** Suppose we minimize LHS first, then go through each production rule one by one considering LHS.

$A \rightarrow C$ , Keep it as it is.

$AC \rightarrow D$ , Here  $A \rightarrow C$  and  $A \rightarrow D$ , So we remove  $A \rightarrow C$ , hence  $A \rightarrow D$  is kept by eliminating C from LHS.

$E \rightarrow AD$ , keep it as it is as E is a single attribute at LHS.

$E \rightarrow H$ , keep it as it is

**Step 2 :** Now we will minimize RHS.

$A \rightarrow C$ , keep it as it is

$A \rightarrow D$ , keep it as it is

$E \rightarrow AD$ . That means  $E \rightarrow A$  and  $E \rightarrow D$ .

As  $A \rightarrow D$  is also present in the FD, so we get  $E \rightarrow A$  and  $A \rightarrow D$ . Thus  $E \rightarrow D$  is transitive. Hence neglect it. So we keep  $E \rightarrow A$  only

$E \rightarrow H$ , Keep it as it is.

**Step 3 :** From steps 1 and 2, we get minimal cover of FD as

$A \rightarrow C$

$A \rightarrow D$

$E \rightarrow A$

$E \rightarrow H$

Hence the canonical for is

$A \rightarrow CD$

$E \rightarrow AH$

**Example 4.12.6** What is functional dependency? Find the minimal cover using the minimal cover algorithm for the following functional dependency

$F = \{AB \rightarrow D, B \rightarrow C, AE \rightarrow B, A \rightarrow D, D \rightarrow EF\}$

**Solution :**

**Step 1 :** We will make right hand sides atomic

$AB \rightarrow D$

$B \rightarrow C$

$AE \rightarrow B$

$A \rightarrow D$

$D \rightarrow E$

$D \rightarrow F$

**Step 2 :** Now we will remove redundant FDs using RHS

- For  $AB \rightarrow D$ . Now compute  $(AB)^+$  without considering the  $AB \rightarrow D$  i.e.  $\{G - (AB \rightarrow D)\}$

We get  $(AB)^+ = \{ABCDEF\}$ . That means we can remove  $AB \rightarrow D$  as it is redundant entry.

Hence grammar is

$B \rightarrow C$

$AE \rightarrow B$

$A \rightarrow D$

$D \rightarrow E$

$D \rightarrow F$

- For  $B \rightarrow C$  compute  $(B)^+$  by considering  $\{G - (B \rightarrow C)\}$

$(B)^+ = \{AEBDF\}$ . As  $C$  is not present in this set. That means  $B \rightarrow C$  is not redundant. So we can not remove it.

Hence grammar is

$B \rightarrow C$

$AE \rightarrow B$

$A \rightarrow D$

$D \rightarrow E$

$D \rightarrow F$

- For  $AE \rightarrow B$ , we will compute  $(AE)^+$  under  $\{G - (AE \rightarrow B)\}$

$(AE)^+ = \{AEDF\}$  as  $B$  is not present in  $(AE)^+$ . So we cannot remove  $AE \rightarrow B$  from grammar. Hence grammar will be

$B \rightarrow C$



$AE \rightarrow B$

$A \rightarrow D$

$D \rightarrow E$

$D \rightarrow F$

For  $A \rightarrow D$ , we will compute  $(A)^+$  under  $(G - (A \rightarrow D))$

$(A)^+ = \{AEBC\}$ . As  $D$  is not present in  $(A)^+$ , hence we can not eliminate  $A \rightarrow D$ . The grammar is

$B \rightarrow C$

$AE \rightarrow B$

$A \rightarrow D$

$D \rightarrow E$

$D \rightarrow F$

- For  $D \rightarrow E$ , compute  $(D)^+$  under  $(G - (D \rightarrow E))$

$(D)^+ = \{DF\}$ . As  $E$  is not present in  $(D)^+$ , We cannot remove  $D \rightarrow E$

- For  $D \rightarrow F$ , compute  $(D)^+$  under  $(G - (D \rightarrow F))$

$(D)^+ = \{DE\}$ . As  $F$  is not present in  $(D)^+$ , We cannot remove  $D \rightarrow F$ . Finally the grammar is

$B \rightarrow C$

$AE \rightarrow B$

$A \rightarrow D$

$D \rightarrow E$

$D \rightarrow F$

### Step 3 : Remove redundant entries based on RHS

$B \rightarrow C$ ,  $A \rightarrow D$ ,  $D \rightarrow E$  and  $D \rightarrow F$  as LHS is atomic.

Now we consider  $AE \rightarrow B$

For  $A$  : compute  $E^+$  with respect to  $(G - (AE \rightarrow B) \cup (E \rightarrow B))$

$E^+$  using  $\{B \rightarrow C, E \rightarrow B, A \rightarrow D, D \rightarrow E, D \rightarrow F\} = EBC$

$E^+$  doesn't contain  $A$ , so  $A$  not redundant in  $AE \rightarrow B$

For  $E$  : compute  $A^+$  with respect to  $(G - (AE \rightarrow B) \cup (A \rightarrow B))$

$A^+$  using  $\{B \rightarrow C, A \rightarrow B, A \rightarrow D, D \rightarrow E, D \rightarrow F\} = ABDEFC$

$A^+$  contains  $E$ , so  $E$  is redundant in  $AE \rightarrow B$

Hence we consider  $AE \rightarrow B$  as  $A \rightarrow B$ .

Finally minimal closure is  $\{B \rightarrow C, A \rightarrow B, A \rightarrow D, D \rightarrow E, D \rightarrow F\}$

**Example 4.12.7** Using the minimal cover algorithm, find the minimal cover for the following FDs :

$F = \{AB \rightarrow C, A \rightarrow D, BD \rightarrow C, D \rightarrow BG, AE \rightarrow F\}.$

**Solution :** We will make right hand side atomic

$AB \rightarrow C$

$A \rightarrow D$

$BD \rightarrow C$

$D \rightarrow B$

$D \rightarrow G$

$AE \rightarrow F$

**Step 2 :** Now we will remove redundant FDs using RHS

- For  $AB \rightarrow C$  we compute  $(AB)^+$  without considering the rule  $AB \rightarrow C$  i.e.  $(G - (AB \rightarrow C))$ . We get  $(AB)^+ = \{ABDBGC\}$  i.e C is present by other way also in the set. Hence we can **remove**  $AB \rightarrow C$  as it is redundant entry.
- For  $A \rightarrow D$ , we compute  $(A)^+$  using  $(G - (A \rightarrow D))$ . We get  $(A)^+ = \{A\}$ . We can not get D. So it is not a redundant entry and we can not remove it.
- For  $BD \rightarrow C$ , We compute  $(BD)^+$  using  $(G - (BD \rightarrow C))$ . We get  $(BD)^+ = \{BDG\}$ . This is also not a redundant entry and we can not remove it.
- For  $D \rightarrow B$ . Let us compute  $(D)^+$  using  $(G - (D \rightarrow B))$ . We get  $(D)^+ = \{DG\}$ . This again indicates that we can not get B without the rule  $D \rightarrow B$ . Hence it is not a redundant entry and we can not remove it.
- Similarly, we can conclude For  $D \rightarrow G$  and  $AE \rightarrow F$  as not redundant entries.
- Finally, the grammar will be

$A \rightarrow D$

$BD \rightarrow C$

$D \rightarrow B$

$D \rightarrow G$

$AE \rightarrow F$

**Step 2 :** Now we will remove redundant entries based on LHS.

The  $A \rightarrow D, D \rightarrow B, D \rightarrow G$  remain as it is in minimal cover as LHS is atomic.

Now consider  $BD \rightarrow C$ , We can replace this by  $B \rightarrow C$  and eliminate  $D$ , as  $D \rightarrow B$  is present.

Similarly consider  $AE \rightarrow F$ . But as we cannot replace it either by  $A \rightarrow F$  or  $E \rightarrow F$ . So it is not redundant.

The minimal cover is  $\{A \rightarrow D, D \rightarrow B, D \rightarrow G, B \rightarrow C, AE \rightarrow F\}$ .

#### 4.13 Algorithms for Decomposition

- Decomposition is the process of **breaking down one table into multiple tables**.
- **Formal definition of decomposition is -**
- A decomposition of relation schema  $R$  consists of replacing the relation schema by two relation schema that each contain a subset of attributes of  $R$  and together include all attributes of  $R$  by storing projections of the instance.
- For example - Consider the following table

Employee\_Department table as follows -

| Eid  | Ename | Age | City      | Salary | Deptid | DeptName       |
|------|-------|-----|-----------|--------|--------|----------------|
| E001 | ABC   | 29  | Pune      | 20000  | D001   | Finance        |
| E002 | PQR   | 30  | Pune      | 30000  | D002   | Production     |
| E003 | LMN   | 25  | Mumbai    | 5000   | D003   | Sales          |
| E004 | XYZ   | 24  | Mumbai    | 4000   | D004   | Marketing      |
| E005 | STU   | 32  | Hyderabad | 25000  | D005   | Human Resource |

We can decompose the above relation Schema into two relation schemas as **Employee** (**Eid, Ename, Age, City, Salary**) and **Department** (**Deptid, Eid, DeptName**) as follows -

##### Employee Table

| Eid  | Ename | Age | City      | Salary |
|------|-------|-----|-----------|--------|
| E001 | ABC   | 29  | Pune      | 20000  |
| E002 | PQR   | 30  | Pune      | 30000  |
| E003 | LMN   | 25  | Mumbai    | 5000   |
| E004 | XYZ   | 24  | Mumbai    | 4000   |
| E005 | STU   | 32  | Hyderabad | 25000  |

## Department Table

| Deptid | Eid  | DeptName       |
|--------|------|----------------|
| D001   | E001 | Finance        |
| D002   | E002 | Production     |
| D003   | E003 | Sales          |
| D004   | E004 | Marketing      |
| D005   | E005 | Human Resource |

- The decomposition is used for eliminating redundancy.
- **For example :** Consider following relation **Schema R** in which we assume that the grade determines the salary, the redundancy is caused

## Schema R

| Name | eid | deptname  | Grade | Salary |
|------|-----|-----------|-------|--------|
| AAA  | 121 | Accounts  | 2     | 8000   |
| AAA  | 132 | Sales     | 3     | 7000   |
| BBB  | 101 | Marketing | 4     | 7000   |
| CCC  | 106 | Purchase  | 2     | 8000   |

- Hence, the above table can be decomposed into two Schema S and T as follow :

| Schema S |     |           |       |
|----------|-----|-----------|-------|
| Name     | eid | deptname  | Grade |
| AAA      | 121 | Accounts  | 2     |
| AAA      | 132 | Sales     | 3     |
| BBB      | 101 | Marketing | 4     |
| CCC      | 106 | Purchase  | 2     |

| Schema T |        |
|----------|--------|
| Grade    | Salary |
| 2        | 8000   |
| 3        | 7000   |
| 4        | 7000   |
| 2        | 8000   |

## Problems related to decomposition :

Following are the potential problems to consider :

- 1) Some queries become more **expensive**.
- 2) Given instances of the decomposed relations, we may not be able to reconstruct the corresponding instance of the original relation!

- 3) Checking some dependencies may require joining the instances of the decomposed relations.
- 4) There may be loss of information during decomposition.

### Properties associated with decomposition

There are two properties associated with decomposition and those are -

- 1) **Loss-less join or non loss decomposition** : When all information found in the original database is preserved after decomposition, we call it as loss less or non loss decomposition.
- 2) **Dependency preservation** : This is a property in which the constraints on the original table can be maintained by simply enforcing some constraints on each of the smaller relations.

#### 4.14 Lossless Join

**SPPU : May-18, Marks 6**

**The lossless join can be defined using following three conditions :**

- i) **Union** of attributes of R1 and R2 must be equal to attribute of R. Each attribute of R must be either in R1 or in R2.

$$\text{Att}(R1) \cup \text{Att}(R2) = \text{Att}(R)$$

- ii) **Intersection** of attributes of R1 and R2 must not be NULL.

$$\text{Att}(R1) \cap \text{Att}(R2) \neq \Phi$$

- iii) **Common attribute** must be a key for at least one relation (R1 or R2)

$$\text{Att}(R1) \cap \text{Att}(R2) \rightarrow \text{Att}(R1)$$

$$\text{or } \text{Att}(R1) \cap \text{Att}(R2) \rightarrow \text{Att}(R2)$$

**Example 4.14.1** Consider the following relation  $R(A, B, C, D)$  and FDs  $A \rightarrow BC$ , is the decomposition of  $R$  into  $R1(A, B, C)$ ,  $R2(A, D)$ . Check if the decomposition is lossless join or not.

**Solution :**

**Step 1 :** Here  $\text{Att}(R1) \cup \text{Att}(R2) = \text{Att}(R)$  i.e  $R1(A,B,C) \cup R2(A,D) = (A,B,C,D)$  i.e  $R$ .

Thus first condition gets satisfied.

**Step 2 :** Here  $R1 \cap R2 = \{A\}$ . Thus  $\text{Att}(R1) \cap \text{Att}(R2) \neq \Phi$ . Here the second condition gets satisfied.

**Step 3 :**  $\text{Att}(R1) \cap \text{Att}(R2) \rightarrow \{A\}$ . Now  $(A)^+ = \{A,B,C\}$  i.e. attributes of R1. Thus the third condition gets satisfied.

This shows that the given decomposition is a **lossless join**.

**Example 4.14.2** Consider the following relation  $R(A, B, C, D, E, F)$  and FDs  $A \rightarrow BC$ ,  $C \rightarrow A$ ,  $D \rightarrow E$ ,  $F \rightarrow A$ ,  $E \rightarrow D$  is the decomposition of  $R$  into  $R1(A, C, D)$ ,  $R2(B, C, D)$  and  $R3(E, F, D)$ . Check for lossless.

**Solution :**

**Step 1 :**  $R1 \cup R2 \cup R3 = R$ . Here the first condition for checking lossless join is satisfied as  $(A,C,D) \cup (B,C,D) \cup (E,F,D) = \{A,B,C,D,E,F\}$  which is nothing but  $R$ .

**Step 2 :** Consider  $R1 \cap R2 = \{CD\}$  and  $R2 \cap R3 = \{D\}$ . Hence second condition of intersection not being  $\Phi$  gets satisfied.

**Step 3 :** Now, consider  $R1(A, C, D)$  and  $R2(B, C, D)$ . We find  $R1 \cap R2 = \{CD\}$

$(CD)^+ = \{ABCDE\} \in$  attributes of  $R1$  i.e.  $\{A, C, D\}$ . Hence condition 3 for checking lossless join for  $R1$  and  $R2$  gets satisfied.

**Step 4 :** Now, consider  $R2(B, C, D)$  and  $R3(E, F, D)$ . We find  $R2 \cap R3 = \{D\}$ .

$(D)^+ = \{D, E\}$  which is neither complete set of attributes of  $R2$  or  $R3$ .

[Note that  $F$  is missing for being attribute of  $R3$ ].

Hence it is not **lossless join decomposition**. Or in other words we can say it is a **lossy decomposition**.

**Example 4.14.3** Suppose that we decompose schema  $R = (A,B,C,D,E)$  into  $(A,B,C)$   $(C,D,E)$  Show that it is not a lossless decomposition.

**Solution :**

**Step 1 :** Here we need to assume some data for the attributes  $A, B, C, D$ , and  $E$ . Using this data we can represent the relation as follows -

**Relation R**

| A | B | C | D | E |
|---|---|---|---|---|
| a | 1 | x | p | q |
| b | 2 | x | r | s |

**Relation R1 = (A,B,C)**

| A | B | C |
|---|---|---|
| a | 1 | x |
| b | 2 | x |

**Relation R2 = (C,D,E)**

| C | D | E |
|---|---|---|
| x | p | q |
| x | r | s |

**Step 2 :** Now we will join these tables using natural join, i.e. the join based on common attribute C. We get  $R1 \bowtie R2$  as

| A | B | C | D | E |
|---|---|---|---|---|
| a | 1 | x | p | q |
| a | 1 | x | r | s |
| b | 2 | x | p | q |
| b | 2 | x | r | s |

Here we get more rows or tuples than original relation R

Clearly  $R1 \bowtie R2 \neq R$ . Hence it is not lossless decomposition.

**Review Question**

1. List the properties of decomposition. Explain lossless join with example.

**SPPU : May-18, End Sem, Marks 6**

**4.15 Dependency Preservation**

- **Definition** : A decomposition  $D = \{R1, R2, R3, \dots, Rn\}$  of R is dependency preserving for a set F of functional dependency if -  $(F1 \cup F2 \cup \dots \cup Fm) = F$ .
- If decomposition is not dependency-preserving, some dependency is lost in the decomposition.

**Example 4.15.1** Consider the relation R (A, B, C) for functional dependency set  $\{A \rightarrow B \text{ and } B \rightarrow C\}$  which is decomposed into two relations  $R1 = (A, C)$  and  $R2 = (B, C)$ . Then check if this decomposition dependency preserving or not.

**Solution :** This can be solved in following steps :

**Step 1 :** For checking whether the decomposition is dependency preserving or not we need to check following condition

$$F^+ = (F1 \cup F2)^+$$

**Step 2 :** We have with us the  $F^+ = \{ A \rightarrow B \text{ and } B \rightarrow C \}$

**Step 3 :** Let us find  $(F1)^+$  for relation R1 and  $(F2)^+$  for relation R2

| R1(A,C)                                          | R2(B,C)                                 |
|--------------------------------------------------|-----------------------------------------|
| A→A Trivial                                      | B→B Trivial                             |
| C→C Trivial                                      | C→C Trivial                             |
| A→C ∴ In (F)+A→B→C and it is Nontrivial          | B→C ∴ In (F) <sup>+</sup> B→C and it is |
| AC→AC Trivial                                    | Non-Trivial                             |
| A→B but is not useful as B is not part of R1 set | BC→BC Trivial                           |
| We can not obtain C→A                            | We can not obtain C→B                   |

**Step 4 :** We will eliminate all the trivial relations and useless relations. Hence we can obtain R1 and R2 as

| R1(A,C)        | R2(B,C)         |
|----------------|-----------------|
| A→C Nontrivial | B→C Non-Trivial |

$$(F1 \cup F2)^+ = \{A \rightarrow C, B \rightarrow C\} \neq \{A \rightarrow B, B \rightarrow C\} \text{ i.e. } (F)^+$$

Thus the condition specified in step 1 i.e.  $F^+ = (F1 \cup F2)^+$  is **not true**. Hence it is **not dependency preserving decomposition**.

**Example 4.15.2** Let relation  $R(A, B, C, D)$  be a relational schema with following functional dependencies  $\{A \rightarrow B, B \rightarrow C, C \rightarrow D \text{ and } D \rightarrow B\}$ . The decomposition of  $R$  into  $(A, B)$ ,  $(B, C)$  and  $(B, D)$ . Check whether this decomposition is dependency preserving or not.

**Solution :**

**Step 1 :** Let  $(F)^+ = \{A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow B\}$ .



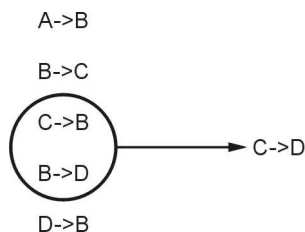
**Step 2 :** We will find  $(F_1)^+$ ,  $(F_2)^+$ ,  $(F_3)^+$  for relations  $R_1(A,B)$ ,  $R_2(B,C)$  and  $R_3(B,D)$  as follows -

| <b>R1(A,B)</b>       | <b>R2(B,C)</b>       | <b>R3(B,D)</b>         |
|----------------------|----------------------|------------------------|
| A→A Trivial          | B→B Trivial          | B→B Trivial            |
| B→B Trivial          | C→C Trivial          | D→D Trivial            |
| A→B ∴ $(F)^+$        | B→C ∴ $(F)^+$ and    | B→D ∴ $(F)^+$ as and   |
| and it's non Trivial | it's non Trivial     | B→C→D and it's non     |
| B→A can not be       | C→B ∴ In $(F)^+$ and | Trivial                |
| obtained             | C→D→C and it is      | D→B ∴ $(F)^+$ and it's |
| AB→AB                | Nontrivial           | non Trivial            |
|                      | BC→BC Trivial        | BD→BD Trivial          |

**Step 3 :** We will eliminate all the trivial relations and useless relations. Hence we can obtain  $R_1 \cup R_2 \cup R_3$  as

| <b>R1(A,B)</b> | <b>R2(B,C)</b> | <b>R3(B,D)</b> |
|----------------|----------------|----------------|
| A→B            | B→C            | B→D            |
|                | C→B            | D→B            |

**Step 4 :** As from above FD's we get



**Step 5 :** This proves that  $F^+ = (F_1 \cup F_2 \cup F_3)^+$ . Hence given **decomposition is dependency preserving**.

**Example 4.15.3** Given relation  $r(X,Y,W,Z,P,Q)$  and the set  $F = \{XY \rightarrow W, XW \rightarrow P, PQ \rightarrow Z, XY \rightarrow Q\}$ . Consider the decomposition  $R_1(Z,P,Q)$ ,  $R_2(X,Y,Z,P,Q)$ . Is this decomposition lossless or lossy? Use lossless join algorithm.

**Solution :**

**Step 1 :** Construct a table with six columns for given six attributes i.e. X, Y, W, Z, P, Q and two rows for given two relation i.e. R1 and R2.

|    | X | Y | W | Z | P | Q |
|----|---|---|---|---|---|---|
| R1 |   |   |   |   |   |   |
| R2 |   |   |   |   |   |   |

**Step 2 :** Fill in the entries as follows -

R1 (Z, P, Q) and R2 (X, Y, Z, P, Q)

Consider R1 having attributes Z, P, Q so put ' $\alpha$ ' in those row and put ' $\beta$ ' other remaining rows same as R2 having attributes X, Y, Z, P, Q so put ' $\alpha$ ' in those row and put ' $\beta$ ' other remaining rows.

|    | X            | Y            | W            | Z          | P          | Q          |
|----|--------------|--------------|--------------|------------|------------|------------|
| R1 | $\beta_{1X}$ | $\beta_{1Y}$ | $\beta_{1W}$ | $\alpha_Z$ | $\alpha_P$ | $\alpha_Q$ |
| R2 | $\alpha_X$   | $\alpha_Y$   | $\beta_{2W}$ | $\alpha_Z$ | $\alpha_P$ | $\alpha_Q$ |

**Step 3 :**

- i) Considering  $XY \rightarrow W$ , we check if the two rows of the table have the same value under the columns XY that make up the determinant of the FD. Since the rows do not have identical values, the table will remain unchanged and we repeat step 3 by considering another FD.
- ii) Considering  $XW \rightarrow P$ , we check if the two rows of the table have the same value under the columns XW that make up the determinant of the FD. Since the rows do not have identical of values, the table will remain unchanged and we repeat step 3 by considering another FD.
- iii) Considering  $PQ \rightarrow Z$ , we check if the two rows of the table have the same value under the columns PQ that make up the determinant of the FD. Since the rows R1 and R2 have values of  $\alpha_P$  and  $\alpha_Q$  under the columns PQ. Therefore, we need to equate all the corresponding entries for the rows under column Z. The values in column Z are already equal no changes are necessary Repeat step 3 again.
- iv) Considering  $XY \rightarrow Q$ , we check if the two rows of the table have the value under the columns XY that make up determinant of the FD. Since rows do not have identical values, the table will remain unchanged. Now, there are no more FDs to consider.

**Step 4 :** Since there is no row in the table that has all  $\Phi$  in it's entries the decomposition is lossy. That is original table cannot be recovered from the join of relations R1 and R2.

**Example 4.15.4** Suppose we decompose the scheme  $R=(A,B,C,D,E)$  into  $(A,B,C)$  and  $(A,D,E)$ .

Show that this decomposition is the lossless decomposition if following functional dependencies hold :  $A \rightarrow BC, CD \rightarrow E, B \rightarrow D$

Show that decomposition is dependency preserving decomposition.

**Solution :**

**Step 1 :** Construct table with five columns for given five attributes i.e. A, B, C, D, E and two rows for given two relations i.e. R1 (A, B, C) and R2 (A, D, E).

|    | A | B | C | D | E |
|----|---|---|---|---|---|
| R1 |   |   |   |   |   |
| R2 |   |   |   |   |   |

**Step 2 :** The attributes of the scheme of R1 are A, B, C. Therefore, we place  $\alpha_A$ ,  $\alpha_B$  and  $\alpha_C$  under these columns respectively. The remaining entries of this row are filled with  $\beta_{1D}$  and  $\beta_{1E}$ . Same as for R2 have attributes A, D, E. Therefore, we place  $\alpha_A$ ,  $\alpha_D$  and  $\alpha_E$  under these columns respectively. The remaining entries of this row are filled with  $\beta_{2B}$  and  $\beta_{2C}$ .

|    | A          | B            | C            | D            | E            |
|----|------------|--------------|--------------|--------------|--------------|
| R1 | $\alpha_A$ | $\alpha_B$   | $\alpha_C$   | $\beta_{1D}$ | $\beta_{1E}$ |
| R2 | $\alpha_A$ | $\beta_{2B}$ | $\beta_{2C}$ | $\alpha_D$   | $\alpha_E$   |

**Step 3 :**

- i) Considering  $A \rightarrow BC$  we check if the two rows of the table have the same value under the columns that make up the determinant of FD. Rows R1 and R2 has values  $\alpha_A$  under column A. Therefore, we need to equate all the corresponding entries for these rows under columns B and C.

Since entry under column B is  $\alpha_B$  (for row R1) and column C is  $\alpha_C$  (for row R2).

|    | A          | B          | C          | D            | E            |
|----|------------|------------|------------|--------------|--------------|
| R1 | $\alpha_A$ | $\alpha_B$ | $\alpha_C$ | $\beta_{1D}$ | $\beta_{1E}$ |
| R2 | $\alpha_A$ | $\alpha_B$ | $\alpha_C$ | $\alpha_D$   | $\alpha_E$   |

- ii) Considering  $CD \rightarrow E$ , we check for rows that have the same value in the columns C and D. Here, rows do not same values, the table remain unchanged and we repeat step 3 by another FD.

- iii) Considering  $B \rightarrow D$ , we check for rows that have the same value in the columns B here rows have same values  $\alpha_B$  under the column B. Therefore, we need to equate all the corresponding entries for these rows under column D. Since entry under column D is  $\alpha_D$  for row R1.

|    | A          | B          | C          | D          | E            |
|----|------------|------------|------------|------------|--------------|
| R1 | $\alpha_A$ | $\alpha_B$ | $\alpha_C$ | $\alpha_D$ | $\beta_{1E}$ |
| R2 | $\alpha_A$ | $\alpha_B$ | $\alpha_C$ | $\alpha_D$ | $\alpha_E$   |

Now there are no more FDs to consider.

**Step 4 :** Since row R2 has become  $\alpha_A, \alpha_B, \alpha_C, \alpha_D, \alpha_E$  i.e. R2 has all  $\alpha$  values. Hence decomposition is lossless.

#### 4.16 Second Normal Form (2NF)

**SPPU : Oct.-19, Marks 7**

Before understanding the second normal form let us first discuss the concept of partial functional dependency and prime and non prime attributes.

#### Concept of partial functional dependency

Partial dependency means that a nonprime attribute is functionally dependent on part of a candidate key.

For example : Consider a relation  $R(A,B,C,D)$  with functional dependency

$\{AB \rightarrow CD, A \rightarrow C\}$

Here (AB) is a candidate key because

$$(AB)^+ = \{ABCD\} = \{R\}$$

Hence {A,B} are prime attributes and {C,D} are non prime attribute. In  $A \rightarrow C$ , the non prime attribute C is dependent upon A which is actually a part of candidate key AB. Hence due to  $A \rightarrow C$  we get partial functional dependency.

#### Prime and non prime attributes

- **Prime attribute :** An attribute, which is a part of the candidate-key, is known as a prime attribute.
- **Non-prime attribute :** An attribute, which is not a part of the prime-key, is said to be a non-prime attribute.
- **Example : Consider a Relation**  $R = \{A,B,C,D\}$  and candidate key as AB, the Prime attributes : A, B.

Non Prime attributes : C, D

## The second normal form

For a table to be in the Second Normal Form, following conditions must be followed

- i) It should be in the First Normal form.
- ii) It should not have partial functional dependency.

For example : Consider following table in which every information about a the Student is maintained in a table such as student id(sid), student name(sname), course id(cid) and course name(cname).

**Student\_Course**

| sid | sname | cid | cname |
|-----|-------|-----|-------|
| 1   | AAA   | 101 | C     |
| 2   | BBB   | 102 | C++   |
| 3   | CCC   | 101 | C     |
| 4   | DDD   | 103 | Java  |

This table is not in 2NF. For converting above table to 2NF we must follow the following steps -

**Step 1 :** The above table is in 1NF.

**Step 2 :** Here **sname** and **sid** are associated similarly **cid** and **cname** are associated with each other. Now if we delete a record with **sid = 2**, then automatically the course C++ will also get deleted. Thus,

**sid->sname** or **cid->cname** is a partial functional dependency, because {**sid,cid**} should be essentially a candidate key for above table. Hence to bring the above table to 2NF we must decompose it as follows :

### Student

| sid | sname | cid |
|-----|-------|-----|
| 1   | AAA   | 101 |
| 2   | BBB   | 102 |
| 3   | CCC   | 101 |
| 4   | DDD   | 103 |

Here candidate key is  
(sid, cid)  
and  
(sid, cid)->sname

**Course**

| cid | cname |
|-----|-------|
| 101 | C     |
| 102 | C++   |
| 101 | C     |
| 103 | Java  |

Here candidate key is cid  
here cid > cname

Thus now table is in 2NF as there is no partial functional dependency.

**Example 4.16.1** Study the relation given below and state what level of normalization can be achieved and normalize it upto that level.

| Order no. | Order date | Item lines |          |            |
|-----------|------------|------------|----------|------------|
|           |            | Item code  | Quantity | Price/Unit |
| 1456      | 26-12-1999 | 3687       | 52       | 50.4       |
|           |            | 4627       | 38       | 60         |
|           |            | 3214       | 20       | 20.00      |
| 1886      | 04-03-1999 | 4629       | 45       | 20.25      |
|           |            | 4627       | 30       | 60.20      |
| 1788      | 04-04-1999 | 4627       | 40       | 60.20      |

**Solution :**

**Reason for the given relation being unnormalized**

1. Observe order for many items.
2. Item lines has many attributes-called composite attributes.
3. Each tuple has variable length.
4. Difficult to store due to non-uniformity.
5. Given item code difficult to find qty-ordered and hence called Unnormalized relation.

**For conversion to first normal form -**

- Identify the composite attributes, convert the composite attributes to individual attributes.
- Duplicate the common attributes as many times as lines in composite attribute.
- Every attribute now describes single property and not multiple properties, some data will be duplicated.
- Now this is called First normal form (1NF) also called flat file.

| Order no. | Order date | Item lines |          |            |
|-----------|------------|------------|----------|------------|
|           |            | Item code  | Quantity | Price/Unit |
| 1456      | 26-12-1999 | 3687       | 52       | 50.4       |
| 1456      | 26-12-1999 | 4627       | 38       | 60         |
| 1456      | 26-12-1999 | 3214       | 20       | 20.00      |
| 1886      | 04-03-1999 | 4629       | 45       | 20.25      |
| 1886      | 04-03-1999 | 4627       | 30       | 60.20      |
| 1788      | 04-04-1999 | 4627       | 40       | 60.20      |

Fig. 4.16.1 Table in first normal form

- The above table has insertion, deletion and update anomalies. For instance - if we delete order no. 1886, then the item code 4629 gets lost. Similarly if we update 4627, then all instances of 4627 need to be changed.
- We need to convert 2NF if it is in 1NF. The non-key attributes are functionally dependent on key attribute and if there is a composite key then no non-key attribute is functionally depend on one part of the key.
- The table can be converted to 2NF as follows -

## Orders

| OrderNo | OrderDate  |
|---------|------------|
| 1456    | 26-12-1999 |
| 1886    | 04-03-1999 |
| 1788    | 04-04-1999 |

## Order details

| OrderNo | ItemCode | Qty |
|---------|----------|-----|
| 1456    | 3687     | 52  |
| 1886    | 4629     | 45  |
| 1788    | 4627     | 40  |

## Prices

| ItemCode | Price/Unit |
|----------|------------|
| 3687     | 50.4       |
| 4627     | 60         |
| 3214     | 20         |
| 4629     | 20.25      |

### Review Question

1. Explain why database normalization is required for good relational database design ? Explain with example requirements of second normal form.

**SPPU : Oct.-19, In Sem, Marks 7**

### 4.17 Third Normal Form (3NF)

**SPPU : Oct.-18, 19, Aug.-17, Dec.-18, Marks 7**

Before understanding the third normal form let us first discuss the concept of transitive dependency, super key and candidate key.

### Concept of transitive dependency

A functional dependency is said to be transitive if it is indirectly formed by two functional dependencies. For example -

$X \rightarrow Z$  is a transitive dependency if the following functional dependencies hold true :

$X \rightarrow Y$

$Y \rightarrow Z$

### Concept of super key and candidate key

**Superkey** : A super key is a set or one of more columns (attributes) to uniquely identify rows in a table.

**Candidate key** : The minimal set of attribute which can uniquely identify a tuple is known as candidate key. For example consider following table

| RegID | RollNo | Sname |
|-------|--------|-------|
| 101   | 1      | AAA   |
| 102   | 2      | BBB   |
| 103   | 3      | CCC   |
| 104   | 4      | DDD   |



## Superkeys

- {RegID}
- {RegID, RollNo}
- {RegID, Sname}
- {RollNo, Sname}
- {RegID, RollNo, Sname}

## Candidate keys

- {RegID}
- {RollNo}

## Third normal form

A table is said to be in the third normal form when,

- i) It is in the second normal form.(i.e. it does not have partial functional dependency).
- ii) It doesn't have transitive dependency.

Or in other words

In other words 3NF can be defined as : A table is in 3NF if it is in 2NF and for each functional dependency

$X \rightarrow Y$

at least one of the following conditions hold :

- i) X is a super key of table.
- ii) Y is a prime attribute of table.

For example : Consider following table **Student\_details** as follows -

| sid | sname | zipcode | cityname | state       |
|-----|-------|---------|----------|-------------|
| 1   | AAA   | 11111   | Pune     | Maharashtra |
| 2   | BBB   | 22222   | Surat    | Gujarat     |
| 3   | CCC   | 33333   | Chennai  | Tamilnadu   |
| 4   | DDD   | 44444   | Jaipur   | Rajasthan   |
| 5   | EEE   | 55555   | Mumbai   | Maharashtra |

Here

**Super keys** : {sid},{sid,sname},{sid,sname,zipcode}, {sid,zipcode,cityname}... and so on.

**Candidate keys :** {sid}

**Non-prime attributes :** {sname,zipcode,cityname,state}

The dependencies can be denoted as

sid->sname

sid->zipcode

zipcode->cityname

cityname->state

The above denotes the transitive dependency. Hence above table is not in 3NF. We can convert it into 3NF as follows :

### Student

| sid | sname | zipcode |
|-----|-------|---------|
| 1   | AAA   | 11111   |
| 2   | BBB   | 22222   |
| 3   | CCC   | 33333   |
| 4   | DDD   | 44444   |
| 5   | EEE   | 55555   |

### Zip

| zipcode | cityname | state       |
|---------|----------|-------------|
| 11111   | Pune     | Maharashtra |
| 22222   | Surat    | Gujarat     |
| 33333   | Chennai  | Tamilnadu   |
| 44444   | Jaipur   | Rajasthan   |
| 55555   | Mumbai   | Maharashtra |

**Example 4.17.1** Consider the relation  $R = \{A, B, C, D, E, F, G, H, I, J\}$  and the set of functional dependencies  $F = \{A, B \rightarrow C, A \rightarrow \{D, E\}, B \rightarrow F, F \rightarrow \{G, H\}, D \rightarrow \{I, J\}\}$

1. What is the key for R ? Demonstrate it using the inference rules.
2. Decompose R into 2NF, then 3NF relations.

**Solution :** Let,

$$A \rightarrow DE \text{ (given)}$$

$$\therefore A \rightarrow D, A \rightarrow E \text{ (decomposition rule)}$$

$$\text{As } D \rightarrow IJ, A \rightarrow IJ$$

Using union rule we get

$$A \rightarrow DEIJ$$

$$\text{As } A \rightarrow A$$

$$\text{we get } A \rightarrow ADEIJ$$

Using augmentation rule we compute AB

$$AB \rightarrow ABDEIJ$$

$$\text{But } AB \rightarrow C \text{ (given)}$$

$$\therefore AB \rightarrow ABCDEIJ$$

$$B \rightarrow F \text{ (given)} \quad F \rightarrow GH \quad \therefore B \rightarrow GH \text{ (transitivity)}$$

$$\therefore AB \rightarrow AGH \text{ is also true}$$

$$\text{Similarly } AB \rightarrow AF \quad \because B \rightarrow F \text{ (given)}$$

Thus now using union rule

$$AB \rightarrow ABCDEFGHIJ$$

$\therefore$  AB is a key

The table can be converted to 2NF as

$$R_1 = (\underline{A}, \underline{B}, C)$$

$$R_2 = (\underline{A}, D, E, I, J)$$

$$R_3 = (\underline{B}, F, G, H)$$

The above 2NF relations can be converted to 3NF as follows

$$R_1 = (\underline{A}, \underline{B}, C)$$

$$R_2 = (\underline{A}, D, E)$$

$$R_3 = (\underline{D}, I, J)$$

$$R_4 = (\underline{B}, E)$$

$$R_5 = (E, G, H).$$

**Example 4.17.2** A software contract and consultancy firm maintains details of all the various projects in which its employees are currently involved. These details comprise :

- Employee number

- *Employee name*
- *Date of birth*
- *Department code*
- *Department name*
- *Project code*
- *Project description*
- *Project supervisor*

*Assume the following :*

- *Each employee number is unique.*
- *Each department has a single department code.*
- *Each project has a single code and supervisor.*
- *Each employee may work on one or more projects.*
- *Employee names need not necessarily be unique.*
- *Project code, project description and project supervisor are repeating fields.*

*Normalise this data to third normal form.*

**Solution :**

### **Un-Normalized Form**

Employee Number, Employee Name \_Date of Birth \_Department Code \_Department Name \_Project Code \_Project Description \_Project Supervisor

### **1NF**

Employee Number, Employee Name \_Date of Birth  
Department Code, Department Name  
Employee Number, Project Code, Project Description \_Project Supervisor

### **2NF**

Employee Number, Employee Name \_Date of Birth \_Department Code \_Department Name  
Employee Number, Project Code,  
Project Code, Project Description, Project Supervisor

### **3NF**

Employee Number, Employee Name \_Date of Birth \_\*Department Code  
Department Code, Department Name  
Employee Number, Project Code  
Project Code, Project Description, Project Supervisor

**Example 4.17.3** What is normalization ? Normalize below given relation upto 3NF  
STUDENT.

| StudID | StudName | City  | Pincode | ProjectID | ProjectName | Course      | Content              |
|--------|----------|-------|---------|-----------|-------------|-------------|----------------------|
| S101   | Ajay     | Surat | 326201  | P101      | Health      | Programming | C++,<br>Java, C      |
| S102   | Vijay    | Pune  | 325456  | P102      | Social      | WEB         | HTML,<br>PHP,<br>ASP |

**Solution :** For converting the given schema to first normal form, we will arrange it in such a way that have each tuple contains single record. For that purpose we need to split the schema into two tables namely **Student** and **Projects**.

### 1NF

#### Student

| StudID | StudName | Pincode | City  |
|--------|----------|---------|-------|
| S101   | Ajay     | 326201  | Surat |
| S102   | Vijay    | 325456  | Pune  |

#### Projects

| StudID | ProjectID | ProjName | Course      | Content |
|--------|-----------|----------|-------------|---------|
| S101   | P101      | Health   | Programming | C++     |
| S101   | P101      | Health   | Programming | Java    |
| S101   | P101      | Health   | Programming | C       |
| S102   | P102      | Social   | WEB         | HTML    |
| S102   | P102      | Social   | WEB         | PHP     |
| S102   | P102      | Social   | WEB         | ASP     |

**2NF**

For a table to be in 2NF, there should not be any partial dependency.

**Student**

| StudID | StudName | Pincode | City  |
|--------|----------|---------|-------|
| S101   | Ajay     | 326201  | Surat |
| S102   | Vijay    | 325456  | Pune  |

**Project**

| StudID | ProjectID | ProjName | CourseID |
|--------|-----------|----------|----------|
| S101   | P101      | Health   | C101     |
| S101   | P101      | Health   | C102     |
| S101   | P101      | Health   | C103     |
| S102   | P102      | Social   | C104     |
| S102   | P102      | Social   | C105     |
| S102   | P102      | Social   | C106     |

**CourseDetails**

| CourseID | Course      | Content |
|----------|-------------|---------|
| C101     | Programming | C++     |
| C102     | Programming | Java    |
| C103     | Programming | C       |
| C104     | WEB         | HTML    |
| C105     | WEB         | PHP     |
| C106     | WEB         | ASP     |

**3NF**

There was a transitive dependency in 2NF tables because city is associated with student ID and city depends upon zip code. Hence the transitive dependency is removed to covert table into 3NF. The required 3NF schema is as below -

**Student**

| StudID | StudName | Pincode |
|--------|----------|---------|
| S101   | Ajay     | 326201  |
| S102   | Vijay    | 325456  |

**Student\_Address**

| Pincode | City  |
|---------|-------|
| 326201  | Surat |
| 325456  | Pune  |

**Project**

| StudID | ProjectID | ProjName | CourseID |
|--------|-----------|----------|----------|
| S101   | P101      | Health   | C101     |
| S101   | P101      | Health   | C102     |
| S101   | P101      | Health   | C103     |
| S102   | P102      | Social   | C104     |
| S102   | P102      | Social   | C105     |
| S102   | P102      | Social   | C106     |

**CourseDetails**

| CourseID | Course      | Content |
|----------|-------------|---------|
| C101     | Programming | C++     |
| C102     | Programming | Java    |
| C103     | Programming | C       |
| C104     | WEB         | HTML    |

|      |     |     |
|------|-----|-----|
| C105 | WEB | PHP |
| C106 | WEB | ASP |

**Example 4.17.4** What is the need for normalization ? Consider the relation :  $Emp\text{-}proj = \{ssn, Pnumber, Hours, Ename, Pname, Plocation\}$   
 Assume  $\{ssn, Pnumber\}$  as primary key.  
 The dependencies are:  
 $\{ssn, Pnumber\} \rightarrow Hours$   
 $Ssn \rightarrow Ename$   
 $Pnumber \rightarrow \{Pname, Plocation\}$   
 Normalize the above relation to 3NF.

**Solution :** Need for normalization - Refer section 4.9.

Consider the given dependencies

- (1)  $\{ssn, Pnumber\} \rightarrow Hours$
- (2)  $ssn \rightarrow Ename$
- (3)  $Pnumber \rightarrow \{Pname, Plocation\}$

The dependencies 2 and 3 represents the partial dependency. Hence we convert the relation into second normal form by splitting the given  $Emp\text{-}proj$  into three relations

$Emp = \{\underline{ssn}, Ename\}$

$Proj = \{\underline{Pnumber}, Pname, Plocation\}$

$Works = \{\underline{ssn}, \underline{Pnumber}, Hours\}$

**Example 4.17.5** Consider the following relation for  $CARSALE(CAR\text{-}NO, Date\text{-}Sold, Salesman\text{-}no, Commission, Discount)$ .

Assume a car can be sold by multiple salesman and hence primary is  $(CAR\text{-}NO, Salesman\text{-}no)$

Additional dependencies are

$Date\_Sold \rightarrow Discount$

$Salesman\_no \rightarrow Commission$

i) Is this relation in 1NF, 2NF, 3NF ? Why and Why not ?

ii) How would you normalize this completely ?

**Solution :**

- i) Let us check the database schema against each normal form.



**First normal form :** As the relation have no multivalued attributes or nested relations, the given relation is in 1<sup>st</sup> normal form.

**Second normal form :** This relation is not in second normal form because the attribute **commission** is dependent on part of primary key **Salesman-no**.

**Third normal form :** This relation is not in third normal form because firstly it is not in 2<sup>nd</sup> normal form and there should be transitive dependency of a nonkey attribute on primary key.

(CAR-No, Salesman-no) → Date\_sold → Discount

(ii) To normalize this relation we will decompose it into

R1={CAR-No, Salesman-no, Date\_sold}

R2={Date\_sold, Discount}

R3={Salesman-no, commission}

The functional dependency is as follows -

F1={(CAR-No, Salesman-no) → Date\_sold}

F2={Date\_sold → Discount}

F3={Salesman-no → commission}

**Example 4.17.6** Normalize the below relation upto 3NF

| Module | Dept | Lecturer | Text |
|--------|------|----------|------|
| M1     | D1   | L1       | T1   |
| M1     | D1   | L1       | T2   |
| M2     | D1   | L1       | T1   |
| M2     | D1   | L1       | T3   |
| M3     | D1   | L2       | T4   |
| M4     | D2   | L3       | T1   |
| M4     | D2   | 3        | T5   |
| M5     | D2   | L4       | T6   |

**Solution :** The given relation is already in 1<sup>st</sup> normal form. But it has Insert, delete and update anomalies. Because -

**1) Insert anomalies :** We can not add a module(M) with no texts(T).

**2) Delete anomalies :** If we remove M3, we remove L2 as well.

**3) Update anomalies :** To change lecturer for M1, we have to change two rows.

Hence we will convert it to second normal form.

**Step 1 :** We can define the functional dependency FD as

{Module, Text}->{Lecturer, Dept}

But

{Module}->{Lecturer, Dept}

That means Lecturer and Dept are partially dependent on the primary key. Hence for conversion of first normal form to second normal form we will decompose the give table into two tables as

**Table 2a**

| Module | Dept | Lecturer |
|--------|------|----------|
| M1     | D1   | L1       |
| M2     | D1   | L1       |
| M3     | D1   | L2       |
| M4     | D2   | L3       |
| M5     | D2   | L4       |

**Table 2b**

| Module | Text |
|--------|------|
| M1     | T1   |
| M1     | T2   |
| M2     | T1   |
| M2     | T3   |
| M3     | T4   |
| M4     | T1   |
| M4     | T5   |
| M5     | T6   |

The relation is now in second normal form.

**Step 3 :** The table 2a has Insert, Delete and Update anomalies. Because -

**1) INSERT anomalies :** We can't add lecturers who teach no modules.

2) **UPDATE anomalies** : To change the department for L1 we must alter two rows.

3) **DELETE anomalies** : If we delete M3 we delete L2 as well.

Hence, to eliminate these anomalies, we decompose table 2a into two tables and convert it to third normal form.

**Step 4** : Hence we get

**Table 3a**

| Lecturer | Dept. |
|----------|-------|
| L1       | D1    |
| L2       | D1    |
| L3       | D2    |
| L4       | D2    |

**Table 3b**

| Module | Lecturer |
|--------|----------|
| M1     | L1       |
| M2     | L1       |
| M3     | L2       |
| M4     | L3       |
| M5     | L4       |

**Step 5** : Thus now the complete relation is decomposed into three tables and it is in third normal form. It is summarized as below

**Table 3a**

| Lecturer | Dept. |
|----------|-------|
| L1       | D1    |
| L2       | D1    |
| L3       | D2    |
| L4       | D2    |

**Table 3b**

| Module | Lecturer |
|--------|----------|
| M1     | L1       |
| M2     | L1       |
| M3     | L2       |
| M4     | L3       |
| M5     | L4       |

**Table 2b**

| Module | Text |
|--------|------|
| M1     | T1   |
| M1     | T2   |
| M2     | T1   |
| M2     | T3   |
| M3     | T4   |
| M4     | T1   |
| M4     | T5   |
| M5     | T6   |

**Example 4.17.7** *Students\_Detail (Stud\_id, Stud\_name, Zip, City)*

Consider above schema, check whether it is in 3NF, if not justify and propose the schema in 3NF.

**SPPU : Oct.-18, In Sem, Marks 5**

**Solution :** The given schema is not in 3NF because there exists following transitive dependencies.

Because city is associated with Stud\_id and city depends upon the zip code. When this dependency is removed then the table will be in 3NF. It is as follows –

**Student**

| Stud_id | Stud_name | Zip |
|---------|-----------|-----|
|         |           |     |

**Student\_address**

| Zip | City |
|-----|------|
|     |      |

**Example 4.17.8** *Consider a table having structure student (Roll\_no, Branch\_code, Marks\_obtained, Exam\_name, Total\_marks).*

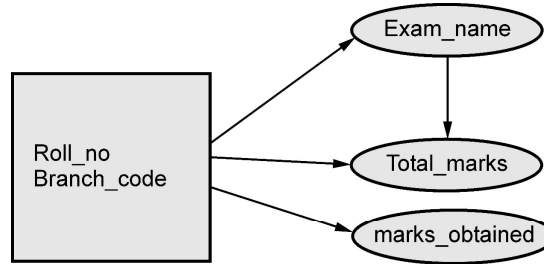
Note following points :

- i) Composite primary key for student table is (Roll\_no, Branch\_code).
- ii) Branch\_code column stores the code of branch for which students have taken admission.
- iii) Exam name attribute is depend on both roll\_no and branch\_code.
- iv) Total marks attribute is depend on exam\_name attribute.

Considering above requirement state whether the table created is in third normal form or not ? Why ? If not in third normal for propose the database design for above requirements which is in third normal form.

**SPPU : Oct.-19, In Sem, Marks 7**

**Solution :** Clearly the above table is not there in third normal form as there exists transitive dependencies. This is because, **exam\_name** attribute depend upon the primary key **roll\_no** and **branch\_code** and **total\_marks** are associated with **exam\_name** attribute.



To convert the above schema to third normal form we need to decompose the schema into different relations –

#### Student\_info

|         |             |           |
|---------|-------------|-----------|
| Roll_no | branch_code | Exam_name |
|---------|-------------|-----------|

#### Exam\_info

|           |             |                |
|-----------|-------------|----------------|
| Exam_name | Total_marks | Marks_obtained |
|-----------|-------------|----------------|

### Review Questions

1. Explain what is normalization ? Explain with example requirements of Third Normal Form.

**SPPU : Aug.-17, In Sem, Marks 5**

2. Explain why Database normalization is required for good relational database design ? Explain with example requirements of different normal forms like 1NF, 2NF and 3NF.

**SPPU : Dec.-18, End Sem, Marks 5**

### 4.18 BCNF

**SPPU ; May-18, Dec.-18,19, Marks 5**

Boyce and Codd Normal Form is a **higher version** of the Third Normal form. This form deals with certain type of anomaly that is not handled by 3NF.

A 3NF table which **does not have multiple overlapping** candidate keys is said to be in BCNF.

Or in other words,

For a table to be in BCNF, following conditions must be satisfied :

- i) R must be in 3<sup>rd</sup> Normal Form
- ii) For each functional dependency (  $X \rightarrow Y$  ), X should be a super key. In simple words if Y is a prime attribute then X can not be non prime attribute.

For example - Consider following table that represents that a student enrollment for the course -

**Enrollment table**

| sid | Course | Teacher |
|-----|--------|---------|
| 1   | C      | Ankita  |
| 1   | Java   | Poonam  |
| 2   | C      | Ankita  |
| 3   | C++    | Supriya |
| 4   | C      | Archana |

From above table following observations can be made :

- One student can enroll for multiple courses. For example student with sid = 1 can enroll for C as well as Java.
- For each course, a teacher is assigned to the student.
- There can be multiple teachers teaching one course for example course C can be taught by both the teachers namely - Ankita and Archana.
- The candidate key for above table can be (sid, course), because using these two columns we can find,
- The above table holds following dependencies
  - (sid, course)→Teacher
  - Teacher→course
- The above table is not in BCNF because of the dependency teacher→course. Note that the teacher is not a superkey or in other words, teacher is a non prime attribute and course is a prime attribute and non-prime attribute derives the prime attribute.
- To convert the above table to BCNF we must decompose above table into student and course tables

**Student**

| sid | Teacher |
|-----|---------|
| 1   | Ankita  |
| 1   | Poonam  |
| 2   | Ankita  |
| 3   | Supriya |
| 4   | Archana |

**Course**

| Teacher | Course |
|---------|--------|
| Ankita  | C      |
| Poonam  | Java   |
| Ankita  | C      |
| Supriya | C++    |
| Archana | C      |

Now the table is in BCNF

**Example 4.18.1** Consider a relation(A,B,C,D) having following FDs.{AB->C, AB->D, C->A, B->D}. Find out the normal form of R.

**Solution :**

**Step 1 :** We will first find out the candidate key from the given FD.

$$(AB)^+ = \{ABCD\} = R$$

$$(BC)^+ = \{ABCD\} = R$$

$$(AC)^+ = \{AC\} \neq R$$

There is no involvement of D on LHS of the FD rules. Hence D can not be part of any candidate key. Thus we obtain two candidate keys **(AB)<sup>+</sup>** and **(BC)<sup>+</sup>**. Hence

**prime attributes** = {A,B,C}

**Non prime attributes** = {D}

**Step 2 :** Now, we will start checking from reverse manner, that means from BCNF, then 3NF, then 2NF.

**Step 3 :** For R being in BCNF for X->Y the X should be candidate key or super key.

From above FDs consider C->D in which C is not a candidate key or super key. Hence given relation is not in BCNF.

**Step 4 :** For R being in 3NF for X->Y either i) the X should be candidate key or super key or

ii) Y should be prime attribute. (For prime and non prime attributes refer step 1)

- For AB->C or AB->D the AB is a candidate key. Condition for 3NF is satisfied.

- Consider  $C \rightarrow A$ . In this FD the C is not candidate key but A is a prime attribute. Condition for 3NF is satisfied.
- Now consider  $B \rightarrow D$ . In this FD, the B is not candidate key, similarly D is not a prime attribute. Hence condition for 3NF fails over here.

Hence given relation is not in 3NF.

**Step 5 :** For R being in 2NF following condition **should not occur**.

Let  $X \rightarrow Y$ , if X is a proper subset of candidate key and Y is a non prime attribute. This is a case of partial functional dependency.

For relation to be in 2NF there should not be any partial functional dependency.

- For  $AB \rightarrow C$  or  $AB \rightarrow D$  the AB is a **complete candidate key**. Condition for 2NF is satisfied.
- Consider  $C \rightarrow A$ . In this FD the C is not candidate key. Condition for 2NF is satisfied.
- Now consider  $B \rightarrow D$ . In this FD, the B is a part of candidate key (AB or BC), similarly D is not a prime attribute. That means partial functional dependency occurs here.

Hence condition for 2NF fails over here.

Hence given relation is not in 2NF.

Therefore we can conclude that the given relation **R is in 1NF**.

**Example 4.18.2** Consider a relation  $R(ABC)$  with following FD  $A \rightarrow B$ ,  $B \rightarrow C$  and  $C \rightarrow A$ . What is the normal form of R ?

**Solution :**

**Step 1 :** We will find the candidate key

$$(A)^+ = \{ABC\} = R$$

$$(B)^+ = \{ABC\} = R$$

$$(C)^+ = \{ABC\} = R$$

Hence A, B and C all are candidate keys

**Prime attributes** = {A,B,C}

**Non prime attribute**{}

**Step 2 :** For R being in BCNF for  $X \rightarrow Y$  the X should be candidate key or super key.



From above FDs

- Consider  $A \rightarrow B$  in which A is a candidate key or super key. Condition for BCNF is satisfied.
- Consider  $B \rightarrow C$  in which B is a candidate key or super key. Condition for BCNF is satisfied.
- Consider  $C \rightarrow A$  in which C is a candidate key or super key. Condition for BCNF is satisfied.

This shows that the given relation **R is in BCNF**.

**Example 4.18.3** Consider table  $R(A,B,C,D,E)$  with FDs as  $A \rightarrow B$ ,  $BC \rightarrow E$ , and  $ED \rightarrow A$ . The table is in which normal form ? Justify your answer.

**Solution :**

**Step 1 :** We will first find out the candidate keys for given relation R

$$(ACD)^+ = \{A,B,C,D,E\}$$

$$(BCD)^+ = \{A,B,C,D,E\}$$

$$(CDE)^+ = \{A,B,C,D,E\}$$

**Step 2 :** Let  $A \rightarrow B$ , the ACD is candidate key and A is a partial key, B is a prime attribute(i.e. it is also part of candidate key). Hence  $A \rightarrow B$  is not a partial functional dependency.

Similarly in  $BC \rightarrow E$  and  $ED \rightarrow A$ ,

E and A are prime-attributes and hence both are not partial functional dependencies.

Hence R is in 2NF.

**Step 3 :** According to 3NF, every non-prime attribute must be dependent on the candidate key.

In the given functional dependencies, all dependent attributes are prime-attributes. Hence the relation R is in 3NF.

**Step 4 :** For R being in BCNF for  $X \rightarrow Y$  the X should be candidate key or super key.

The table is not in BCNF, none of A, BC and ED contain a key.

**Example 4.18.4** A college maintains details of its lecturers' subject area skills. These details comprise :

- Lecturer number
- Lecturer name
- Lecturer grade
- Department code
- Department name
- Subject code
- Subject name
- Subject level

Assume that each lecturer may teach many subjects but may not belong to more than one department.

Subject code, subject name and subject level are repeating fields.

Normalise this data to third normal form.

**Solution :**

### Unnormalized form

Lecturer Number, Lecturer Name, Lecturer Grade, Department Code, Department Name, Subject Code, Subject Name, Subject Level

### 1NF

|                |               |                |                 |                 |
|----------------|---------------|----------------|-----------------|-----------------|
| LecturerNumber | Lecturer Name | Lecturer Grade | Department Code | Department Name |
|----------------|---------------|----------------|-----------------|-----------------|

|                        |                     |              |               |
|------------------------|---------------------|--------------|---------------|
| <u>Lecturer Number</u> | <u>Subject Code</u> | Subject Name | Subject Level |
|------------------------|---------------------|--------------|---------------|

### 2NF

|                 |                |                 |                  |                  |
|-----------------|----------------|-----------------|------------------|------------------|
| Lecturer Number | _Lecturer Name | _Lecturer Grade | _Department Code | _Department Name |
|-----------------|----------------|-----------------|------------------|------------------|

|                        |                     |
|------------------------|---------------------|
| <u>Lecturer Number</u> | <u>Subject Code</u> |
|------------------------|---------------------|

|                     |               |                |
|---------------------|---------------|----------------|
| <u>Subject Code</u> | _Subject Name | _Subject Level |
|---------------------|---------------|----------------|

**3NF**

|                        |               |                |
|------------------------|---------------|----------------|
| <u>Lecturer Number</u> | Lecturer Name | Lecturer Grade |
|------------------------|---------------|----------------|

|                        |                  |
|------------------------|------------------|
| <u>Department Code</u> | _Department Name |
|------------------------|------------------|

|                        |                     |
|------------------------|---------------------|
| <u>Lecturer Number</u> | <u>Subject Code</u> |
|------------------------|---------------------|

|                     |              |               |
|---------------------|--------------|---------------|
| <u>Subject Code</u> | Subject Name | Subject Level |
|---------------------|--------------|---------------|

**Example 4.18.5** *Prove that any relational schema with two attributes is in BCNF.*

**Solution :** Here, we will consider  $R=\{A,B\}$  i.e. a relational schema with two attributes. Now various possible FDs are  $A \rightarrow B$ ,  $B \rightarrow A$ .

From the above FDs

- Consider  $A \rightarrow B$  in which A is a candidate key or super key. Condition for BCNF is satisfied.
- Consider  $B \rightarrow A$  in which B is a candidate key or super key. Condition for BCNF is satisfied.
- Consider both  $A \rightarrow B$  and  $B \rightarrow A$  with both A and B is candidate key or super key. Condition for BCNF is satisfied.
- No FD holds in relation R. In this  $\{A,B\}$  is candidate key or super key. Still condition for BCNF is satisfied.

This shows that any relation R **is in BCNF** with two attributes.

**Example 4.18.6** *Prove the statement "Every relation which is in BCNF is in 3NF but the converse is not true."*

**Solution :** For a relations to be in 3NF

A table is said to be in the Third Normal Form when,

- i) It is in the Second Normal form. (i.e. it does not have partial functional dependency)
- ii) It doesn't have transitive dependency.

Or in other words

In other words 3NF can be defined as : A table is in 3NF if it is in 2NF and for each functional dependency  $X \rightarrow Y$

at least one of the following conditions hold :

iii)  $X$  is a super key of table

iv)  $Y$  is a prime attribute of table

For a relation to be in BCNF

1) It should be in 3NF

2) A 3NF table which **does not have multiple overlapping** candidate keys is said to be in BCNF.

For proving that the table can be in 3NF but not in BCNF consider Following relation

$R(\text{Student, Subject, Teacher})$  . Consider following are FDs

**$(\text{Subject, Student}) \rightarrow \text{Teacher}$**

Because subject and student combination gives unique teacher.

**$\text{Teacher} \rightarrow \text{Subject}$**

Because each teacher teaches only Subject.

**$(\text{Teacher, Student}) \rightarrow \text{Subject}$**

- So, this relation is in 3NF as every non-key attribute is non-transitively fully functional dependent on the primary key.
- But it is not in BCNF. Because this is a case of **overlapping of candidate keys** because there are two composite candidate keys :
  - $(\text{Subject, Student})$
  - $(\text{Teacher, Student})$

And student is a common attribute in both the candidate keys.

So we need to normalize the above table to BCNF. For that purpose we must set Teacher to be a candidate key

The decomposition of above takes place as follows

$R1(\text{Student, Teacher})$

$R2(\text{Teacher, Subject})$

Now table is in 3NF, as well as in BCNF.

This show that the relation Every relation which is in BCNF is in 3NF but the converse is not true.

**Example 4.18.7** Consider relational schema  $R(A,B,C)$  with FD's  $AB \rightarrow C$  and  $C \rightarrow A$ . Show that schema  $R$  is in 3NF but not in BCNF. Determine minimal keys of  $R$ .

**Solution :** A table is said to be in 3<sup>rd</sup> normal form when,

- i) It is in 2<sup>nd</sup> normal form.
- ii) It doesn't have transitive dependency or in other words L.H.S must be a candidate key or R.H.S is prime attribute.

**For 1<sup>st</sup> Condition :**

**Step 1 : For 2<sup>nd</sup> Normal form condition :**

**Step A :** It is in 1<sup>st</sup> normal form

**Step B :** Table not contain partial dependency i.e. all the non-prime attributes should be fully functionally dependent on candidate key.

Given relation  $R(A, B, C)$

FD :  $AB \rightarrow C, C \rightarrow A$

We will first find out the candidate key from the given FD.

$$(AB)^+ = \{ABC\} = R,$$

$$(CB)^+ = \{ABC\} = R,$$

$$(C)^+ = \{CA\} \neq R$$

They we obtain  $(AB)^+$  and  $(CB)^+$  candidate key.

Hence, prime attributes =  $\{ABC\}$

None prime attributes =  $\{C\}$

Hence, For  $AB \rightarrow C$

$AB$  is candidate key and its present at R.H.S for  $C \rightarrow A$ .

$C$  is also candidate key and its present at R.H.S. There is no partial dependency. Hence it is in 2<sup>nd</sup> normal form.

**Step 2 :** Candidate key =  $(AB)^+$  and  $(CB)^+$

Prime attributes  $(ABC)$

Non prime attribute ( ) Null

For given DF's  $AB \rightarrow C, C \rightarrow A$

In  $AB \rightarrow C$ , L.H.S is AB i.e. it is candidate key and In  $C \rightarrow A$ , L.H.S is C i.e. it is not candidate key but A is present at R.H.S i.e. it is prime attributes.

Hence, given relation is present in 3NF.

Now find given relation is present in BCNF or not.

**Step 3 :** Boyce / Codd normal form (BCNF) :

For a table to be in BCNF, following conditions must be satisfied.

i) R must be in 3<sup>rd</sup> normal form

ii) For each functional dependency ( $X \rightarrow Y$ ), X should be a super key or candidate key.

In above 1<sup>st</sup> condition becomes true it is in 3<sup>rd</sup> normal form. Now check for 2<sup>nd</sup> condition. From given FDs

$AB \rightarrow C$  and  $C \rightarrow A$  in which AB is candidate key but C is not candidate key. Hence 2<sup>nd</sup> condition becomes false so given relation is not present in BCNF.

**Example 4.18.8** What is the difference between 3NF and BCNF ?

**Solution :**

| Sr. No. | 3NF                                                                                                                                                                                        | BCNF                                                                                                            |
|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| 1.      | 3NF stands for Third Normal Form.                                                                                                                                                          | BCNF stands for Boyce Codd Normal Form.                                                                         |
| 2.      | The table is in 3NF if it is in 2NF and for each functional dependency $X \rightarrow Y$ at least following condition hold:<br>(i) X is a superkey,<br>(ii) Y is prime attribute of table. | The table is in BCNF if it is in 3rd normal form and for each relation $X \rightarrow Y$ X should be super key. |
| 3.      | 3NF can be obtained without sacrificing all dependencies.                                                                                                                                  | Dependencies may not be preserved in BCNF.                                                                      |
| 4.      | Lossless decomposition can be achieved in 3NF.                                                                                                                                             | Lossless decomposition is hard to obtain in BCNF.                                                               |
| 5.      | 3NF can be achieved without losing any information from the old table.                                                                                                                     | For obtaining BCNF we may lose some information from old table.                                                 |

**Example 4.18.9** Consider following relation -

*Book(book\_title, authurname, book\_type, listprice, author\_affiliation, publisher)*

Suppose the following functional dependencies exist –

*book\_title* → *publisher, book\_type*

*book\_type* → *listprice*

*authurname* → *author\_affiliation*

i) What normal form is the relation in ? Explain your answer.

ii) Apply normalization until you can not decompose the relations further. State the reasons behind each decomposition.

**Solution :**

- (i) The key for this relation is (Book\_title, Authurname). This relation is in 1NF and not in 2NF as no attributes are Fully Functionally Dependent on the key.

**Step 1 :** We will first decompose it in **2<sup>nd</sup> Normal Form** by following decomposition.

R1(book\_title, authurname)

R2(book\_title, publisher, book\_type, listprice)

R3(authurname, authoraffiliation)

**Reason :** This decomposition eliminates partial functional dependencies.

**Step 2 :** Now we will decompose it further to bring the table in **3<sup>rd</sup> Normal form**

R1(book\_title, authurname)

R2(book\_title, publisher, book\_type, listprice)

R21(book\_title, publisher, book\_type)

R22(book\_type, listprice)

R3(authurname, authoraffiliation)

**Reason :** This decomposition eliminates transitive functional dependency of listprice.

### Review Question

1. Explain 3NF and BCNF. Also enlist their differences.

**SPPU : May-18, Dec.-18,19, End Sem, Marks 5**

### Unit - III

## Multiple Choice Questions

- Q.1** Foreign key is one in which the \_\_\_\_ of one relation is referenced in another relation.
- |                                          |                                        |
|------------------------------------------|----------------------------------------|
| <input type="checkbox"/> a candidate key | <input type="checkbox"/> b constraint  |
| <input type="checkbox"/> c primary key   | <input type="checkbox"/> d foreign key |
- Q.2** The relationship between the two tables are created using \_\_\_\_.
- |                                          |                                              |
|------------------------------------------|----------------------------------------------|
| <input type="checkbox"/> a primary key   | <input type="checkbox"/> b check constraints |
| <input type="checkbox"/> c candidate key | <input type="checkbox"/> d foreign key       |
- Q.3** Constraints are preferred methods for enforcing \_\_\_\_.
- |                                             |                                           |
|---------------------------------------------|-------------------------------------------|
| <input type="checkbox"/> a data abstraction | <input type="checkbox"/> b data access    |
| <input type="checkbox"/> c data inheritance | <input type="checkbox"/> d data integrity |
- Q.4** A table that displays data redundancies yields \_\_\_\_ anomalies.
- |                                      |                                         |
|--------------------------------------|-----------------------------------------|
| <input type="checkbox"/> a insertion | <input type="checkbox"/> b deletion     |
| <input type="checkbox"/> c update    | <input type="checkbox"/> d all of these |
- Q.5** The different classes of relations created by the technique for preventing modification anomalies are called \_\_\_\_.
- |                                                    |                                                              |
|----------------------------------------------------|--------------------------------------------------------------|
| <input type="checkbox"/> a normal forms            | <input type="checkbox"/> b referential integrity constraints |
| <input type="checkbox"/> c functional dependencies | <input type="checkbox"/> d none of the above                 |
- Q.6** For some relations, changing the data can have undesirable consequences called \_\_\_\_.
- |                                                              |                                                    |
|--------------------------------------------------------------|----------------------------------------------------|
| <input type="checkbox"/> a referential integrity constraints | <input type="checkbox"/> b modification anomalies  |
| <input type="checkbox"/> c normal forms                      | <input type="checkbox"/> d transitive dependencies |
- Q.7** The three rules that are used to find the logically implied functional dependencies are called as \_\_\_\_.
- |                                             |                                          |
|---------------------------------------------|------------------------------------------|
| <input type="checkbox"/> a closure rules    | <input type="checkbox"/> b Codd's rules  |
| <input type="checkbox"/> c Armstrong's rule | <input type="checkbox"/> d none of these |



**Q.8** FD stands for \_\_\_\_.

- |                                                  |                                                  |
|--------------------------------------------------|--------------------------------------------------|
| <input type="checkbox"/> a Functional Dependency | <input type="checkbox"/> b Facilitate Dependency |
| <input type="checkbox"/> c Functional Data       | <input type="checkbox"/> d Facilitate Data       |

**Q.9** Which key is referencing a primary key in a table ?

- |                                        |                                          |
|----------------------------------------|------------------------------------------|
| <input type="checkbox"/> a Primary key | <input type="checkbox"/> b Candidate key |
| <input type="checkbox"/> c Foreign key | <input type="checkbox"/> d All of these  |

**Q.10** If  $K \rightarrow R$  then K is said to be the \_\_\_\_\_ of R.

- |                                          |                                          |
|------------------------------------------|------------------------------------------|
| <input type="checkbox"/> a candidate key | <input type="checkbox"/> b foreign key   |
| <input type="checkbox"/> c super key     | <input type="checkbox"/> d none of these |

**Q.11** If there is more than one key for relation schema in DBMS then each key in relation schema is classified as \_\_\_\_.

- |                                          |                                        |
|------------------------------------------|----------------------------------------|
| <input type="checkbox"/> a prime key     | <input type="checkbox"/> b super key   |
| <input type="checkbox"/> c candidate key | <input type="checkbox"/> d primary key |

**Q.12** The form of dependency in which the set of attributes that are neither a subset of any of the keys nor the candidate key is classified as \_\_\_\_.

- |                                                  |                                                        |
|--------------------------------------------------|--------------------------------------------------------|
| <input type="checkbox"/> a transitive dependency | <input type="checkbox"/> b full functional dependency  |
| <input type="checkbox"/> c partial dependency    | <input type="checkbox"/> d prime functional dependency |

**Q.13** The property of normalization of relations which guarantees that functional dependencies are represented in separate relations after decomposition is classified as \_\_\_\_.

- |                                                              |
|--------------------------------------------------------------|
| <input type="checkbox"/> a nonadditive join property         |
| <input type="checkbox"/> b independency reservation property |
| <input type="checkbox"/> c dependency preservation property  |
| <input type="checkbox"/> d additive join property            |

**Q.14**  $X \rightarrow Y$  is trivial if \_\_\_\_.

- |                                            |                                          |
|--------------------------------------------|------------------------------------------|
| <input type="checkbox"/> a $X \subset Y$   | <input type="checkbox"/> b $Y \subset X$ |
| <input type="checkbox"/> c $X \supseteq Y$ | <input type="checkbox"/> d none of these |

**Q.15** A table has fields F1, F2 , F3 , F4, F5 with the following functional dependencies

$F1 \rightarrow F3$ ,

$F2 \rightarrow F4$

$(F1, F2) \rightarrow F5$

In terms of normalization, this table is in \_\_\_\_.

- |                                |                                          |
|--------------------------------|------------------------------------------|
| <input type="checkbox"/> a 1NF | <input type="checkbox"/> b 2NF           |
| <input type="checkbox"/> c 3NF | <input type="checkbox"/> d none of these |

**Q.16** Which normal form is considered adequate for relational database design ?

- ☐ a 2NF ☐ b 3NF  
☐ c 4NF ☐ d BCNF

**Q.17** Lossless, dependency preserving decomposition into \_\_\_\_ normal form is always possible.

- ☐ a 2NF ☐ b 3NF  
☐ c 4NF ☐ d BCNF

**Q.18** After normalization, the original table can be obtained by \_\_\_\_.

- ☐ a delete operation ☐ b join operation  
☐ c retrieve operation ☐ d cascade operation

**Q.19** FDs are the types of constraints that are based on \_\_\_\_.

- ☐ a data ☐ b tables  
☐ c keys ☐ d none of these

**Q.20** A property which ensures that each functional dependency is represented in some individual relational resulting after decomposition is called \_\_\_\_.

- ☐ a loss less join ☐ b dependency preservation  
☐ c both a and b ☐ d none of the above

**Q.21** F+ is called as \_\_\_\_ of F.

- ☐ a recursion ☐ b closure  
☐ c next version ☐ d none of these

**Q.22** A functional dependency must first satisfy the second normal form to satisfy the third normal form.

- ☐ a True ☐ b False

#### Answers Keys for Multiple Choice Questions :

|      |   |      |   |      |   |      |   |
|------|---|------|---|------|---|------|---|
| Q.1  | c | Q.2  | d | Q.3  | d | Q.4  | d |
| Q.5  | a | Q.6  | b | Q.7  | c | Q.8  | a |
| Q.9  | a | Q.10 | c | Q.11 | c | Q.12 | a |
| Q.13 | c | Q.14 | a | Q.15 | a | Q.16 | b |
| Q.17 | b | Q.18 | b | Q.19 | c | Q.20 | c |
| Q.21 | b | Q.22 | a |      |   |      |   |

